Student Name: _____

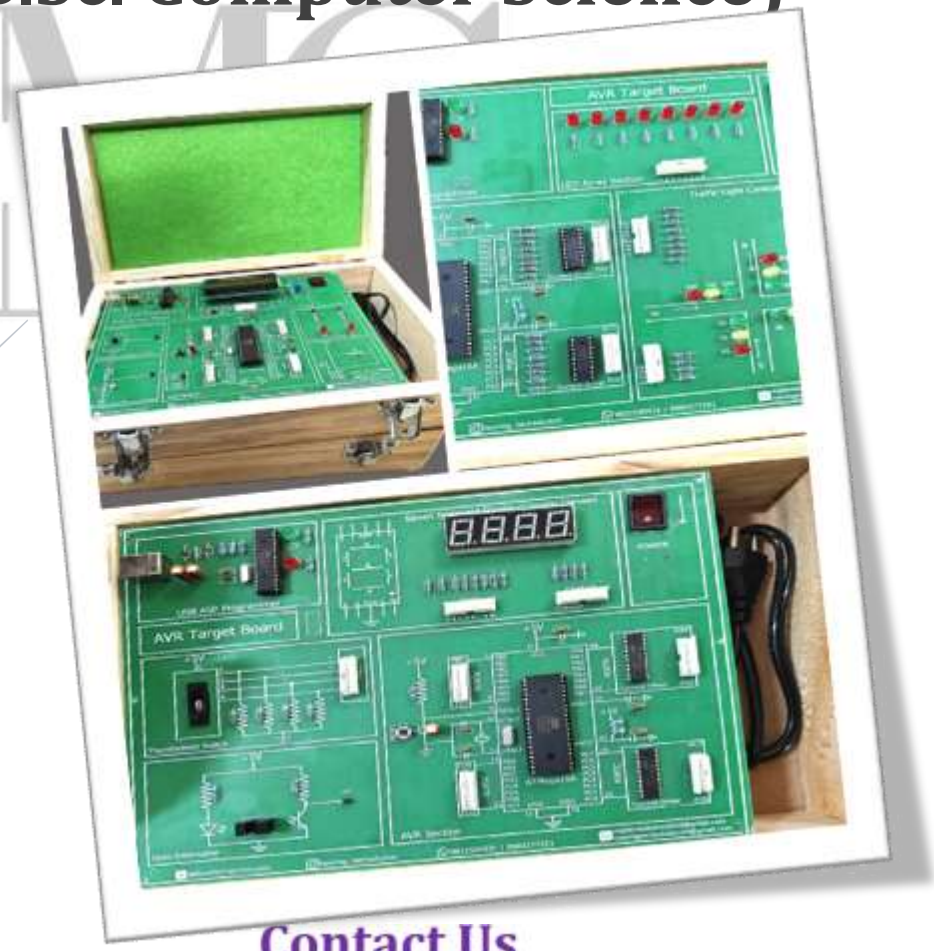
Class: _____ *Batch:* _____

Subject: _____

Lab course on Embedded System with AVR Microcontroller

(Semester - III)

(S.Y.B.Sc. Computer Science)



Contact Us

M.G. TechSolution Sales & Services
Dr. M. A. Shaikh (SET, Ph.D)
Mobile: 9960277201, 9834975744
Email: mgtechsolution2020@gmail.com

New M.G. TechSolution
Mr. G. V. Avhale (SET, NET)
Mobile: 9822590931
Email: newmgtechsolution24@gmail.com

S.Y.B.Sc. Computer Science

INDEX

Lab course on Embedded System with AVR Microcontroller ***Course No. - CS-241-MNP***

Sr. No	Name of the Experiment	Remark & Signature
1.	Study of LED Array interfacing to AVR and program to display various patterns	
2.	Study of Traffic light controller using AVR.	
3.	Study of blinking LED, generate a delay using AVR Timer.	
4.	Study of SSD interfacing and write program for rolling display.	
5.	Study of Event Counter using opto-interrupter and SSD.	
6.	Study of Interfacing thumb wheel switch to AVR and display input data on SSD.	
7.	Study of Intrusion detection security system using IR / PIR sensor and Buzzer.	
8.	Write a program to get the status of the switch and turn on / off LED / buzzer / AC appliances using relay.	
9.	Study to control devices using Smart phone & Bluetooth Module HC05.	
10.	Study of 16*2 LCD interfacing and its programming.	
11.	Study of interfacing LM35 sensor to on chip ADC of AVR & display temp on LCD.	
12.	Study of Light intensity measurement using LDR sensor and AVR.	
13.	Study of Interfacing DHT11 sensor to AVR	
14.	Study of Stepper Motor interfacing and write a program for clockwise and anticlockwise rotation of motor.	
15.	Study of waveform generations using DAC and AVR.	
16.	DC motor interfacing & write a program to vary speed of DC motor using PWM.	

Scan this QR Code
to
Watch Demo Videos

NEWMG_TECHSOLUTION

Scan this QR Code
to
Join Our Instagram Account

Introduction to AVR ATMega16

Introduction:

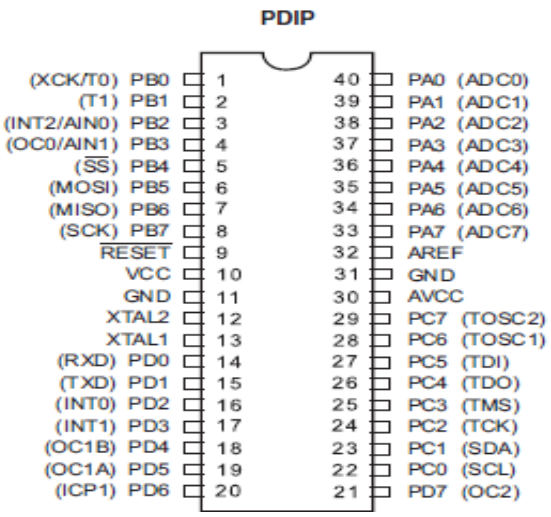
AVR microcontrollers are 8-bit RISC (Reduced Instruction Set Computing) microcontrollers developed by Atmel (now Microchip Technology) that are known for their high performance, low power consumption, and versatility. They are based on a modified Harvard architecture with on-chip flash memory, SRAM, EEPROM, and I/O data space.

AVR microcontrollers offer a combination of performance, low power consumption, and versatility, making them a popular choice for a wide range of applications in embedded systems design.

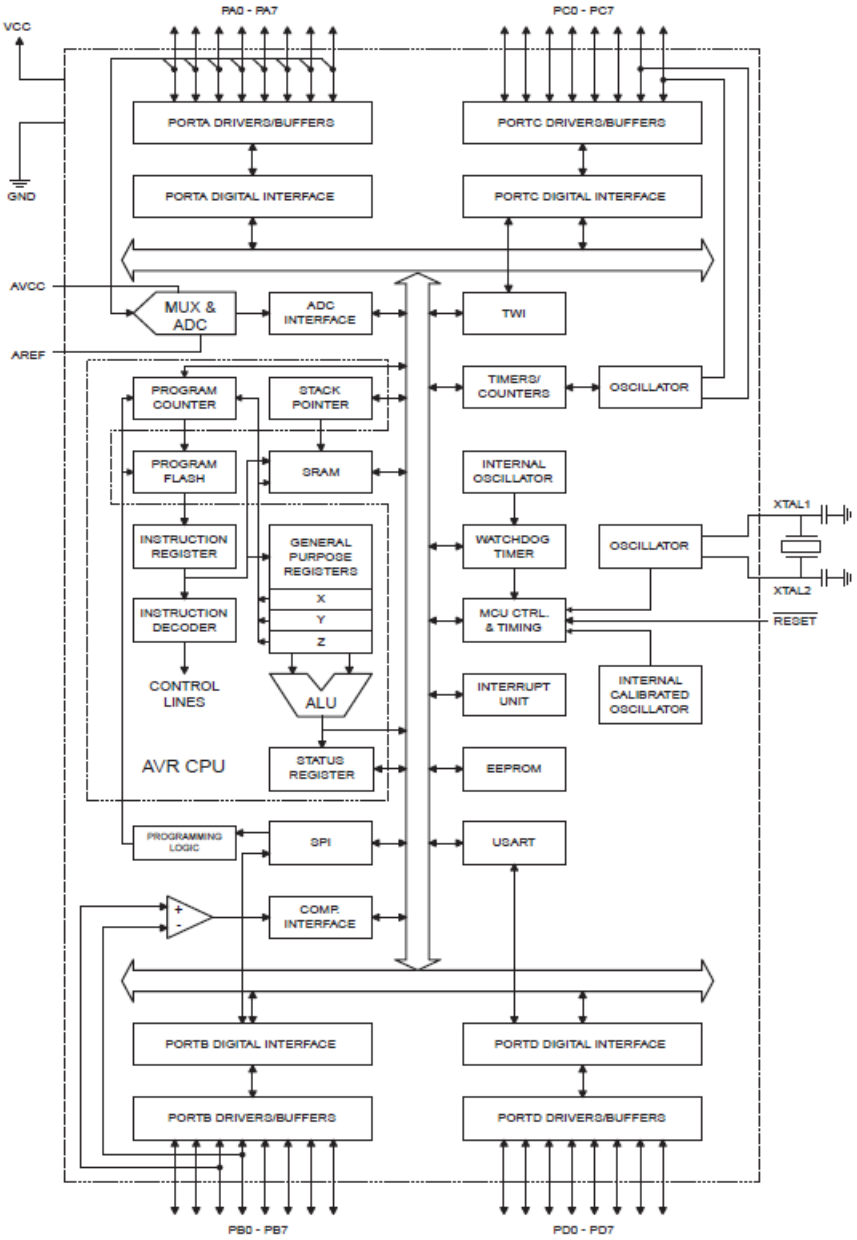
Features of ATmega16:

- High-performance, Low-power AVR® 8-bit Microcontroller
- Advanced RISC Architecture - 131 Powerful Instructions
- 16K Bytes of In-System Self-Programmable Flash
- 512 Bytes EEPROM
- 1K Byte Internal SRAM
- Two 8-bit Timer/Counters with Separate Prescalers and Compare Modes
- One 16-bit Timer/Counter with Separate Prescaler, Compare Mode, and Capture Mode
- 8-channel, 10-bit ADC
- Programmable Serial USART
- Master/Slave SPI Serial Interface
- Byte-oriented Two-wire Serial Interface
- 32 Programmable I/O Lines

Pin Diagram:



Architecture:



Software Tools required for Programming AVR Microcontroller

Microchip Studio Software

- **Software Download Link:**
  https://www.microchip.com/en-us/tools-resources/develop/microchip-studio#Downloads
- **How to Download & Install Microchip Studio Software:** Click on the link below to watch video on Youtube.

  https://youtu.be/poaTbEJrVeU
- **How to use Microchip Studio Software for AVR programming:** Click on the link below to watch video on Youtube.

  https://youtu.be/YyYQUDR6ZLQ

ProgISP Software (used to upload program in AVR)

- **How to use ProgISP Software to upload program in AVR:** Click on the link below to watch video on Youtube.

  https://youtu.be/okQT0lvqJhE

----------**********----------

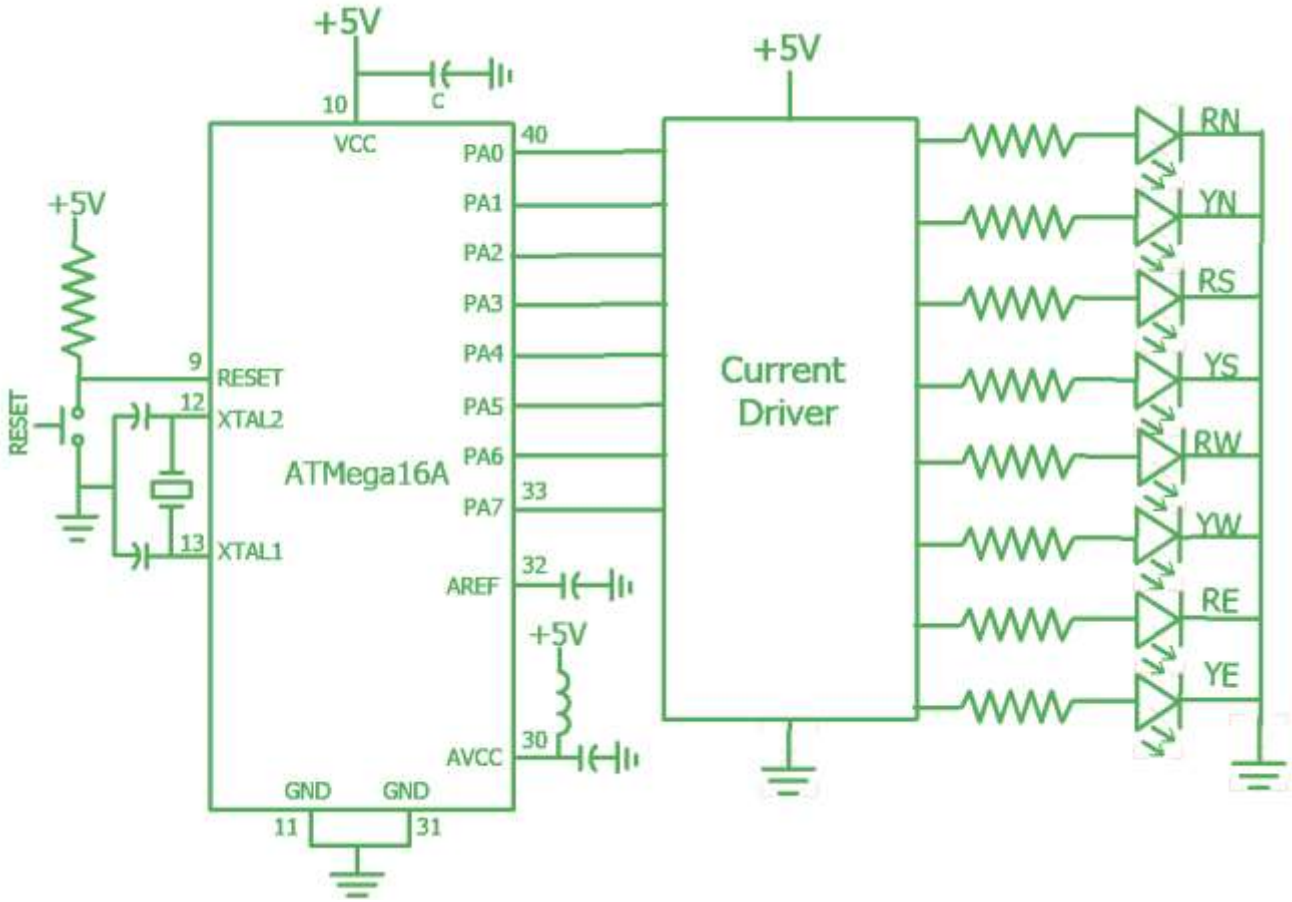
- **Key Feature**
- **USB ASP Programmer available on Every Board.**
- **In- built Power Supply.**
- **Demonstration Videos are available on our Youtube Channel.**
- **Easy to Use and Proper training will be given.**

Experiment – 1

Title: Interfacing of LED array to AVR

Aim: To study interfacing of LED array to AVR microcontroller and program to display various patterns.

Circuit Diagram:



Theory:

LED (Light Emitting Diode): A LED is a two-lead semiconductor light source. It is a p–n junction diode, which emits light when activated. When a suitable voltage is applied to the leads, electrons are able to recombine with electron holes within the device, releasing energy in the form of photons. This effect is called electroluminescence, and the color of the light (corresponding to the energy of the photon) is determined by the energy band gap of the semiconductor. The charge carriers recombine in a forward-biased P-N junction as the electrons cross from the N-region and recombine with the holes existing in the P-region. Free electrons are in the conduction band of energy levels, while holes are in the valence energy band. Thus the energy level of the holes will be lesser than the energy levels of the electrons. Some portion of the energy must be dissipated in order to recombine the electrons and the holes. This energy is emitted in the form of heat and light. The electrons dissipate energy in the form of heat for silicon and germanium diodes but in gallium arsenide phosphide (GaAsP) and gallium

phosphide (GaP) semiconductors, the electrons dissipate energy by emitting photons. If the semiconductor is translucent, the junction becomes the source of light as it is emitted, thus becoming a light-emitting diode, but when the junction is reverse biased no light will be produced by the LED and, on the contrary, the device may also be damaged.

Procedure:

- Start AVR Studio software on PC and write a program.
- Connect the USB cable between USB ASP programmer on the Target board and PC.
- Turn ON the power supply of the target board and then load the .hex file into the AVR using "ProgISP" software.
- Turn OFF the power supply while making circuit connections on the board. Make necessary connections as shown in the circuit diagram.
- Switch ON the power supply of the target board and observe the output.

Program:

Data to be sent to PORTA to turn ON / OFF LED's can be calculated as shown in following table.
(0 = LED OFF, 1 = LED ON)

PA7	PA6	PA5	PA4	PA3	PA2	PA1	PA0	PORTA
D7	D6	D5	D4	D3	D2	D1	D0	Data
Blinking LED's								
0	0	0	0	0	0	0	0	0X00
1	1	1	1	1	1	1	1	0XFF
LED Swaping								
0	0	0	0	1	1	1	1	0X0F
1	1	1	1	0	0	0	0	0XF0
Odd- Even								
0	1	0	1	0	1	0	1	0X55
1	0	1	0	1	0	1	0	0XAA
Running LED's (Right to Left)								
0	0	0	0	0	0	0	1	0X01
0	0	0	0	0	0	1	0	0X02
0	0	0	0	0	1	0	0	0X04
0	0	0	0	1	0	0	0	0X08
0	0	0	1	0	0	0	0	0X10
0	0	1	0	0	0	0	0	0X20
0	1	0	0	0	0	0	0	0X40

1	0	0	0	0	0	0	0	0X80
Running LED's (Left to Right)								
1	0	0	0	0	0	0	0	0X80
0	1	0	0	0	0	0	0	0X40
0	0	1	0	0	0	0	0	0X20
0	0	0	1	0	0	0	0	0X10
0	0	0	0	1	0	0	0	0X08
0	0	0	0	0	1	0	0	0X04
0	0	0	0	0	0	1	0	0X02
0	0	0	0	0	0	0	1	0X01

1. ***Pattern 1 - Blinking LED***

```
#include <avr/io.h>
#define F_CPU 4000000UL
#include <util/delay.h>

void delay(unsigned int value)
{
        unsigned int i;
        for(i=0;i<value;i++)
        {
            _delay_ms(100);
        }
}

int main(void)
{
        DDRA=0xFF;    //make PORTA as o/p
        while (1)
        {
                PORTA = 0x00;
                delay(10);
                PORTA = 0xFF;
                delay(10);
        }
```

```
}
```

2. Pattern 2 – LED Swaping

```
#include <avr/io.h>
#define F_CPU 4000000UL
#include <util/delay.h>

void delay(unsigned int value)
{
	unsigned int i;
	for(i=0;i<value;i++)
	{
		_delay_ms(100);
	}
}

int main(void)
{
	DDRA=0xFF;	//make PORTA as o/p
	while (1)
	{
		PORTA = 0x0F;
		delay(10);
		PORTA = 0xF0;
		delay(10);
	}
}
```

3. Pattern 3 – Odd-Even

```
#include <avr/io.h>
#define F_CPU 4000000UL
#include <util/delay.h>

void delay(unsigned int value)
{
```

```
        unsigned int i;
        for(i=0;i<value;i++)
        {
                _delay_ms(100);
        }
}

int main(void)
{
        DDRA=0xFF;    //make PORTA as o/p
        while (1)
        {
                PORTA = 0x55;
                delay(10);
                PORTA = 0xAA;
                delay(10);
        }
}
```

4. Pattern 4 – Running LED's (Right to Left)

```
#include <avr/io.h>
#define F_CPU 4000000UL
#include <util/delay.h>
int main(void)
{
        DDRA=0xFF;    //make PORTA as o/p
        while (1)
        {
                PORTA = 0x01;
                _delay_ms(150);
                PORTA = 0x02;
                _delay_ms(150);
                PORTA = 0x04;
                _delay_ms(150);
                PORTA = 0x08;
```

```
            _delay_ms(150);
            PORTA = 0x10;
            _delay_ms(150);
            PORTA = 0x20;
            _delay_ms(150);
            PORTA = 0x40;
            _delay_ms(150);
            PORTA = 0x80;
            _delay_ms(150);
        }
}
```

5. Pattern 5 – Running LED's (Left to Right)

```
#include <avr/io.h>
#define F_CPU 4000000UL
#include <util/delay.h>
int main(void)
{
        DDRA=0xFF;    //make PORTA as o/p
        while (1)
        {
            PORTA = 0x80;
            _delay_ms(150);
            PORTA = 0x40;
            _delay_ms(150);
            PORTA = 0x20;
            _delay_ms(150);
            PORTA = 0x10;
            _delay_ms(150);
            PORTA = 0x08;
            _delay_ms(150);
            PORTA = 0x04;
            _delay_ms(150);
            PORTA = 0x02;
            _delay_ms(150);
```

```
            PORTA = 0x01;
            _delay_ms(150);
        }
}
```

TECHSOLUTION
NEW

Result: Successfully studied the Interfacing of LED array with AVR microcontroller and observe the different sequences.

Teacher's Sign

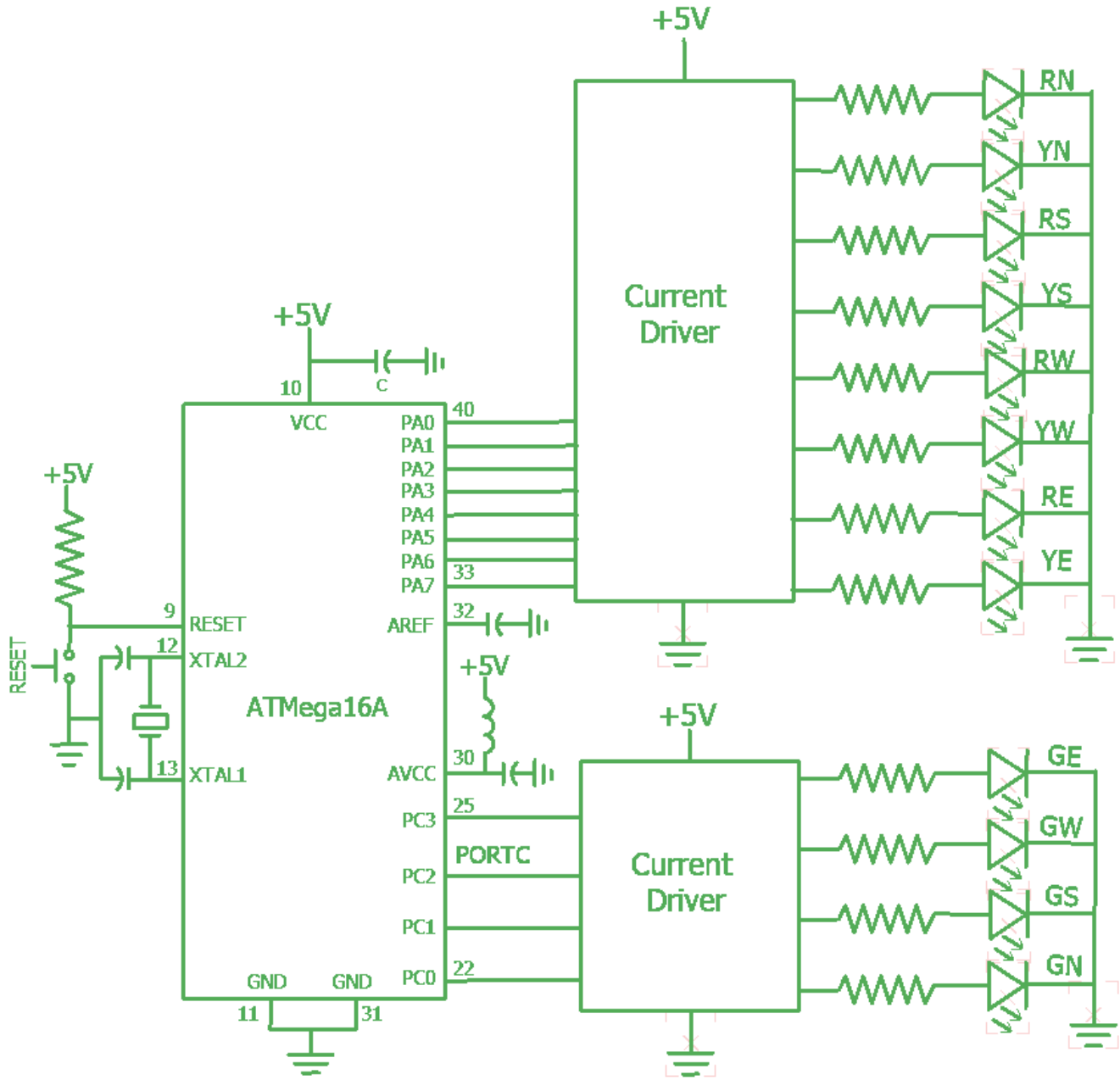
----------*********----------

Experiment – 2

Title: Study of Traffic light controller using AVR

Aim: To study and understand Interfacing Traffic light controller to AVR.

Circuit Diagram:



Theory:

Traffic light controllers are essential for managing the flow of traffic at intersections, ensuring safety and reducing congestion. A microcontroller-based traffic light controller uses a microcontroller, a small computer, to manage traffic lights. The microcontroller is programmed to control the sequence of lights (red, yellow, green) and adjust the timing based on traffic conditions. This provides a centralized, flexible, and efficient way to manage intersections compared to traditional controllers.

Procedure:

- Start AVR Studio software on PC and write a program.
- Connect the USB cable between USB ASP programmer on the Target board and PC.
- Turn ON the power supply of the target board and then load the .hex file into the AVR using "ProgISP" software.
- Turn OFF the power supply while making circuit connections on the board. Make necessary connections as shown in the circuit diagram.
- Switch ON the power supply of the target board and observe the output.

Program:

Data to be sent to PORTA & PORTC to turn ON / OFF traffic light LED's can be calculated as shown in following table. (0 = LED OFF, 1 = LED ON)

PA7	*PA6*	*PA5*	*PA4*	*PA3*	*PA2*	*PA1*	*PA0*	*PORTA*	*PC3*	*PC2*	*PC1*	*PC0*	*PORTC*
YE	*RE*	*YW*	*RW*	*YS*	*RS*	*YN*	*RN*	*Data*	*GE*	*GW*	*GS*	*GN*	*Data*
0	1	0	1	0	1	0	0	0X54	0	0	0	1	0X01
0	1	0	1	0	0	0	1	0X51	0	0	1	0	0X02
0	1	0	0	0	1	0	1	0X45	0	1	0	0	0X04
0	0	0	1	0	1	0	1	0X15	1	0	0	0	0X08

Program 1: Traffic flow – North -> South -> West -> East

```
#include <avr/io.h>
#define F_CPU 4000000UL
#include <util/delay.h>
 void delay(unsigned int value)
 {
        unsigned int i;
        for(i=0; i<value; i++)
        {
                _delay_ms(100);
        }
}
int main(void)
{
   /*diable JTAG port*/
 MCUCSR|=(1<<JTD); // sendig 1 to JTD
```

```
MCUCSR|=(1<<JTD);

  DDRA = 0xFF;   //make PORTA as o/p
  DDRC = 0xFF;   //make PORTC as o/p
   while (1)
   {
          //North
          PORTA = 0X54;
          PORTC = 0X01;
          delay(100);
          //south
          PORTA = 0X51;
          PORTC = 0X02;
          delay(100);
          //west
          PORTA = 0X45;
          PORTC = 0X04;
          delay(100);
          //east
          PORTA = 0X15;
          PORTC = 0X08;
          delay(100);
   }
}
```

Program 2: Traffic flow – Night Mode (only yellow blinking)

PA7	PA6	PA5	PA4	PA3	PA2	PA1	PA0	PORTA	PC3	PC2	PC1	PC0	PORTC
YE	RE	YW	RW	YS	RS	YN	RN	Data	GE	GW	GS	GN	Data
0	0	0	0	0	0	0	0	0X00	0	0	0	0	0X00
1	0	1	0	1	0	1	0	0XAA	0	0	0	0	0X00

```
 #include <avr/io.h>
#define F_CPU 4000000UL
#include <util/delay.h>
 void delay(unsigned int value)
 {
```

```
        unsigned int i;
        for(i=0; i<value; i++)
        {
                _delay_ms(100);
        }
}
int main(void)
{
/*diable JTAG port*/
 MCUCSR|=(1<<JTD); // sendig 1 to JTD
 MCUCSR|=(1<<JTD);

 DDRA = 0xFF;   //make PORTA as o/p
 DDRC = 0xFF;   //make PORTC as o/p
 while (1)
 {
        PORTA = 0X00;
        PORTC = 0X00;
        delay(10);
        PORTA = 0XAA;
        PORTC = 0X00;
        delay(10);
 }
}
```

Result: Interfacing of Traffic light controller with AVR is successfully studied, tested and observe o/p.

Teacher's Sign.

----------**********----------

Experiment – 3

Title: Study of blinking LED, generate a delay using AVR Timer

Aim: To study of blinking LED, generate a delay using AVR Timer.

Circuit Diagram:

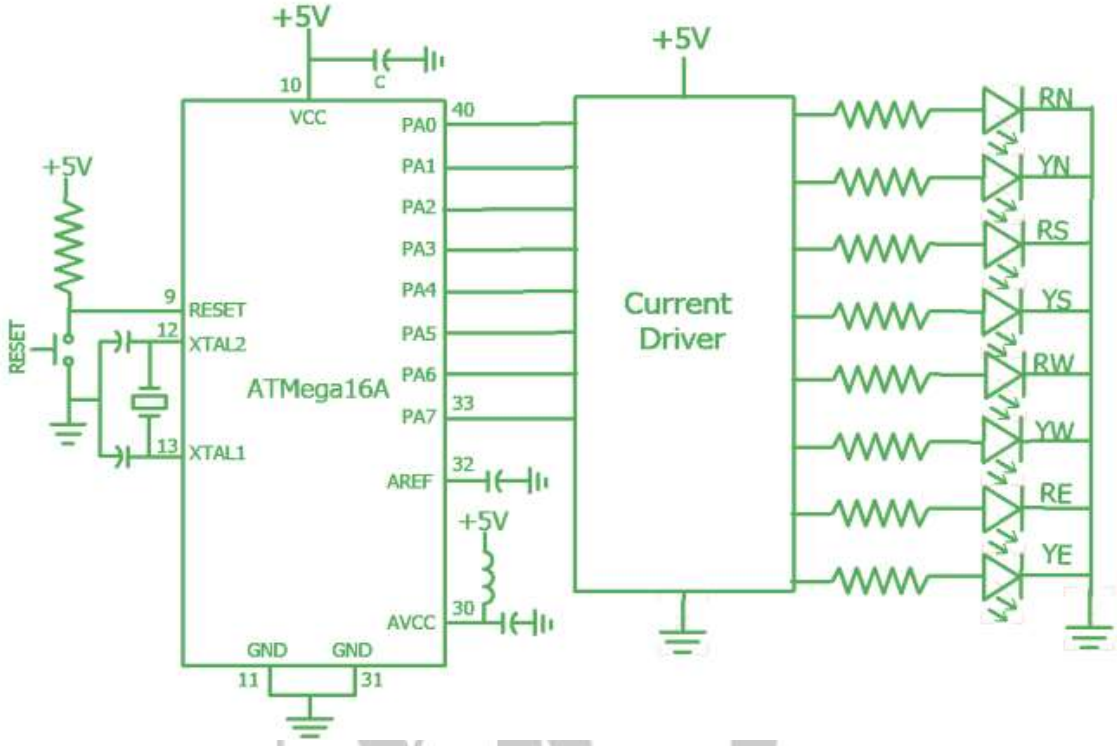


Figure: LED interfacing to AVR

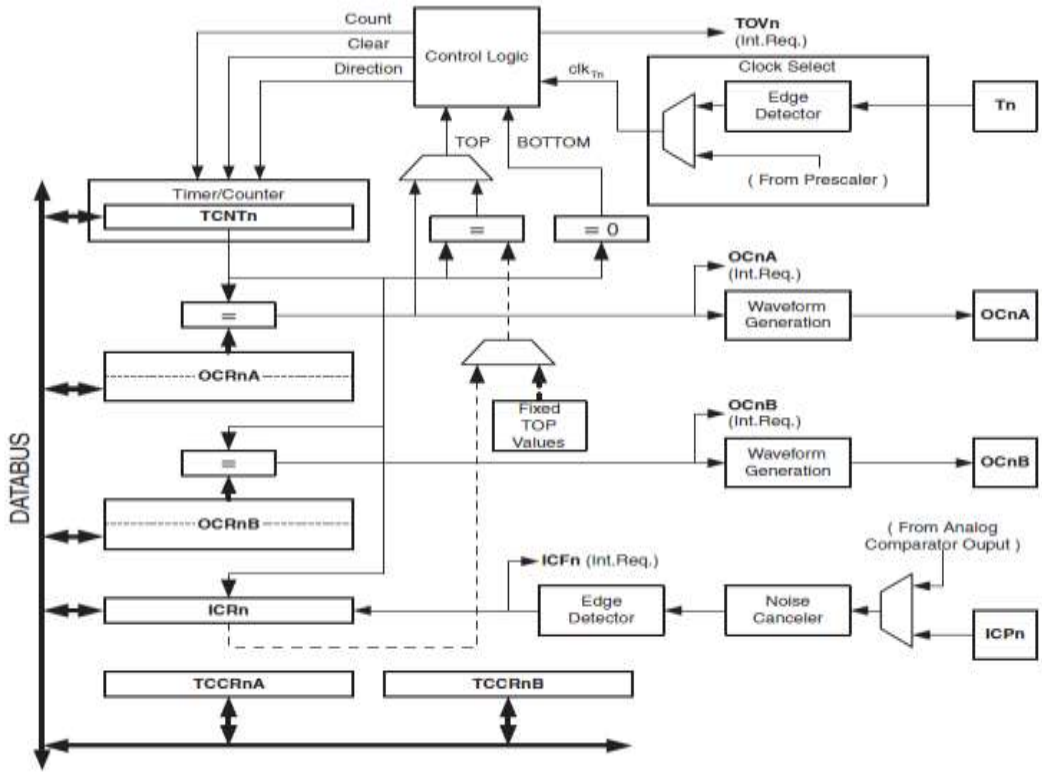


Figure: Timer 1 Block Diagram

Theory:

Every microcontroller have Timer / counter modules in it, that generally used to generate time delays, waveforms, or to count events. Also, the timer is used for PWM generation, capturing events, etc. In AVR ATmega16, there are three timers - Timer0: 8-bit timer, Timer1: 16-bit timer & Timer2: 8-bit timer. In this practical, we use Timer 1 register to generate a delay. For that First, we need to understand the basic registers of the Timer1. The Timer 1 is 16 bit, that means it can count from 0 to 65535. Timer1 also has two control registers named TCCR1A (Timer/counter 1 control register) and TCCR1B. The TOV1 (timer overflow) flag bit goes HIGH when overflow occurs. Timer1 also has the prescaler options of 1:1, 1:8, 1:64, 1:256, and 1:1024. There are two OCR registers in Timer1: OCR1A and OCR1B. There are two separate flags for each of the OCR registers, which act independently of each other. Whenever TCNT1 equals OCR1A, the OCF1A flag will be set on the next timer clock. When TCNT equals OCR1B, the OCF1B flag will be set on the next clock. As Timer1 is a 16-bit timer, the OCR registers are 16-bit registers as well and they are made of two 8-bit registers. For example, OCR1A is made of OCR1AH (OCR1A high byte) and OCR1AL (OCR1A low byte). The TIFR register contains the TOV1, OCF1A, and OCF1B flags.

Bit	7	6	5	4	3	2	1	0
	OCF2	TOV2	ICF1	OCF1A	OCF1B	TOV1	OCF0	TOV0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Initial Value	0	0	0	0	0	0	0	0

TOV0	D0	Timer0 overflow flag bit
	0 = Timer0 did not overflow.	
	1 = Timer0 has overflowed (going from $FF to $00).	
OCF0	D1	Timer0 output compare flag bit
	0 = compare match did not occur.	
	1 = compare match occurred.	
TOV1	D2	Timer1 overflow flag bit
OCF1B	D3	Timer1 output compare B match flag
OCF1A	D4	Timer1 output compare A match flag
ICF1	D5	Input Capture flag
TOV2	D6	Timer2 overflow flag
OCF2	D7	Timer2 output compare match flag

Figure: TIFR (Timer/Counter Interrupt Flag Register) Register

Bit	7	6	5	4	3	2	1	0
	COM1A1	COM1A0	COM1B1	COM1B0	FOC1A	FOC1B	WGM11	WGM10
Read/Write	R/W	R/W	R	R/W	R/W	R/W	R/W	R/W
Initial Value	0	0	0	0	0	0	0	0

COM1A1:COM1A0 D7 D6 Compare Output Mode for Channel A (discussed in Section 3)

COM1B1:COM1B0 D5 D4 Compare Output Mode for Channel B (discussed in Section 3)

FOC1A D3 Force Output Compare for Channel A (discussed in Section 3)

FOC1B D2 Force Output Compare for Channel B (discussed in Section 3)

WGM11:10 D1 D0 Timer1 mode (discussed in Figure 18)

Figure: TCCR1A (Timer 1 Control) Register

Bit	7	6	5	4	3	2	1	0	
	ICNC1	ICES1	-	WGM13	WGM12	CS12	CS11	CS10	TCCR1B
Read/Write	R/W	R/W	R	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

ICNC1 D7 Input Capture Noise Canceler
0 = Input Capture is disabled.
1 = Input Capture is enabled.

ICES1 D6 Input Capture Edge Select
0 = Capture on the falling (negative) edge
1 = Capture on the rising (positive) edge

D5 Not used

WGM13:WGM12 D4 D3 Timer1 mode

Mode	WGM13	WGM12	WGM11	WGM10	Timer/Counter Mode of Operation	Top	Update of OCR1x	TOV1 Flag Set on
0	0	0	0	0	Normal	0xFFFF	Immediate	MAX
1	0	0	0	1	PWM, Phase Correct, 8-bit	0x00FF	TOP	BOTTOM
2	0	0	1	0	PWM, Phase Correct, 9-bit	0x01FF	TOP	BOTTOM
3	0	0	1	1	PWM, Phase Correct, 10-bit	0x03FF	TOP	BOTTOM
4	0	1	0	0	CTC	OCR1A	Immediate	MAX
5	0	1	0	1	Fast PWM, 8-bit	0x00FF	TOP	TOP
6	0	1	1	0	Fast PWM, 9-bit	0x01FF	TOP	TOP
7	0	1	1	1	Fast PWM, 10-bit	0x03FF	TOP	TOP
8	1	0	0	0	PWM, Phase and Frequency Correct	ICR1	BOTTOM	BOTTOM
9	1	0	0	1	PWM, Phase and Frequency Correct	OCR1A	BOTTOM	BOTTOM
10	1	0	1	0	PWM, Phase Correct	ICR1	TOP	BOTTOM
11	1	0	1	1	PWM, Phase Correct	OCR1A	TOP	BOTTOM
12	1	1	0	0	CTC	ICR1	Immediate	MAX
13	1	1	0	1	Reserved	-	-	-
14	1	1	1	0	Fast PWM	ICR1	TOP	TOP
15	1	1	1	1	Fast PWM	OCR1A	TOP	TOP

CS12:CS10 D2D1D0 Timer1 clock selector

0 0 0	No clock source (Timer/Counter stopped)
0 0 1	clk (no prescaling)
0 1 0	clk / 8
0 1 1	clk / 64
1 0 0	clk / 256
1 0 1	clk / 1024
1 1 0	External clock source on T1 pin. Clock on falling edge.
1 1 1	External clock source on T1 pin. Clock on rising edge.

Figure: TCCR1B (Timer 1 Control) Register

OCR1A Value Calculation:

Fosc = 4MHz, Pre-scalar value selected = 1024

So the timer clock source frequency = 4MHz / 1024 = 3906.25 Hz

Time Period of 1 cycle = 256 uS

(hence, maximum delay can be generated using Timer1 is 65536 X 256uS = 16.77 Sec)

e.g. 1) Suppose we want generate delay of 100 mS

Therefore, for a delay of 100 ms, the OCR1A Value required = 100 mS / 256uS = 390

So, we need to save OCR1A with the value 390 to generate delay of 100 mS.

e.g. 2) Suppose we want generate delay of 1000 mS = 1 sec

Therefore, for a delay of 1000 ms, the OCR1A Value required = 1000 mS / 256uS = 3906

So, we need to save OCR1A with the value 3906 to generate delay of 1000 mS.

Procedure:

- Start AVR Studio software on PC and write a program.
- Connect the USB cable between USB ASP programmer on the Target board and PC.
- Turn ON the power supply of the target board and then load the .hex file into the AVR using "ProgISP" software.
- Turn OFF the power supply while making circuit connections on the board. Make necessary connections as shown in the circuit diagram.
- Switch ON the power supply of the target board and observe the output.

Program:

```
#include<avr/io.h>
void T1delay (unsigned int delay)                    //delay value must be in mili-second
{
        float OCR_value = (float) delay / 0.256;
        OCR1A = (unsigned int) OCR_value;
        TCNT1 = 0;
        while((TIFR & (1<<OCF1A)) == 0);     // wait till the timer overflow flag is SET
        TIFR |= (1<<OCF1A) ;                 //clear timer1 overflow flag
}

int main()
{
        DDRA = 0xFF;        //configure led as outpout
        TCCR1B = (1<<WGM12) | (1<<CS10) | (1<<CS12);   //set the pre-scalar as 1024
```

```
        while(1)
        {
                PORTA = 0x00;
                T1delay(100);          //delay value must be in mili-second
                PORTA = 0xFF;
                T1delay(100);          //delay value must be in mili-second
        }
}
```

Calculations:

TECHSOLUTION
NEW

Result: Successfully studied the use of AVR Timer to generate delay and make LED on/off for different delays.

Teacher's Sign

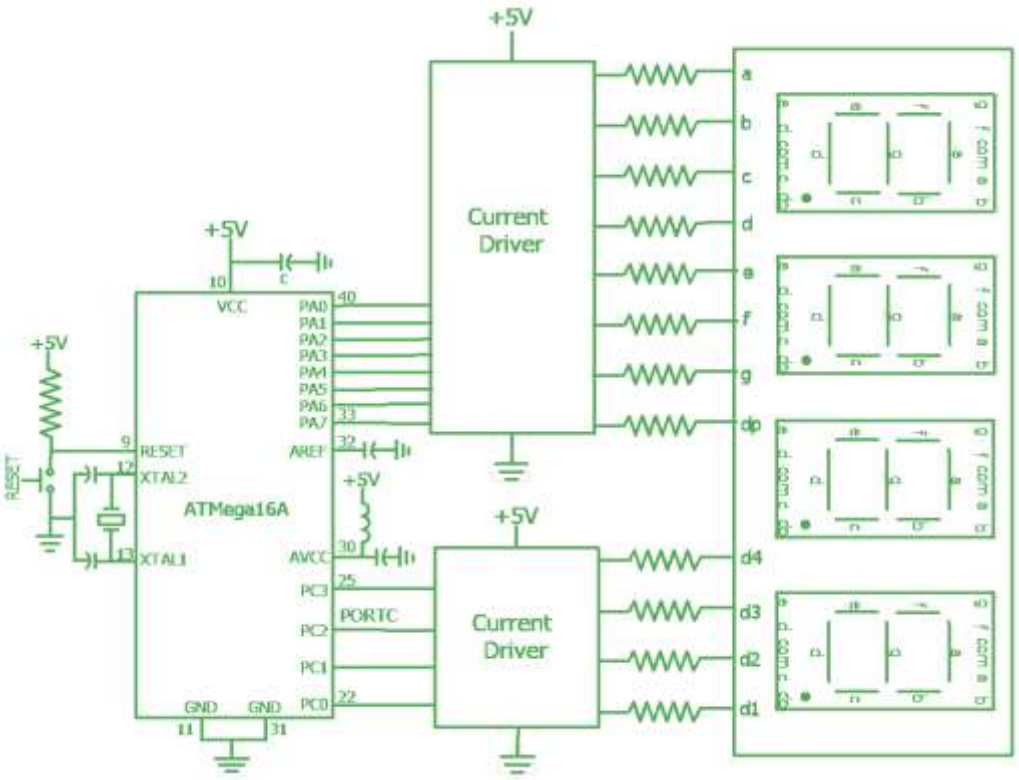
----------*********----------

Experiment – 4

Title: Study of SSD interfacing and write program for rolling display

Aim: To study interfacing of SSD to AVR and write a program for rolling display.

Circuit Diagram:



Theory:

A seven-segment display (SSD), is a form of electronic display device for displaying decimal numerals that is an alternative to the more complex dot matrix displays. The seven elements of the display can be lit in different combinations to represent the Arabic numerals. Often the seven segments are arranged in an oblique (slanted) arrangement, which aids readability. In most applications, the seven segments are of nearly uniform shape and, though in the case of adding machines, the vertical segments are longer and more oddly shaped at the ends in an effort to further enhance readability. The segments of a 7-segment display are referred to by the letters A to G, where the optional decimal point (an "eighth segment", referred to as DP) is used for the display of non-integer numbers.

In a simple LED package, typically all of the cathodes or all of the anodes of the segment LEDs are connected and brought out to a common pin; this is referred to as a "common cathode" or "common anode" display. Hence a 7 segment plus decimal point package will only require nine pins, though commercial products typically contain more pins, and/or spaces where pins would go, in order to match standard IC sockets. Integrated displays also exist, with single or multiple digits. Seven segment displays are used to display 0-9 nos. Also hexadecimal digits can be displayed on seven-segment displays.

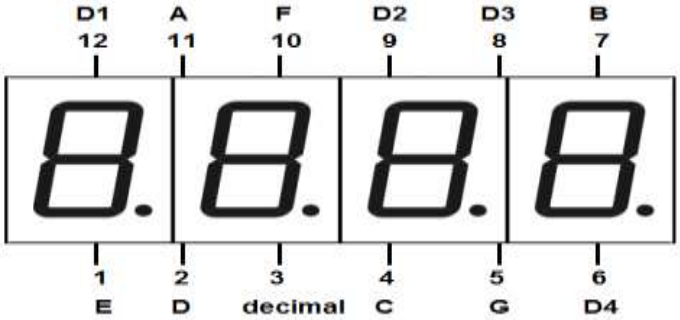


Fig. Four Digit Seven Segment Display (Common Cathode) Pinout

Procedure:

- Start AVR Studio software on PC and write a program.
- Connect the USB cable between USB ASP programmer on the Target board and PC.
- Turn ON the power supply of the target board and then load the .hex file into the AVR using "ProgISP" software.
- Turn OFF the power supply while making circuit connections on the board. Make necessary connections as shown in the circuit diagram.
- Switch ON the power supply of the target board and observe the output.

Program:

Data to be transferred to display 0-9 numbers on common cathode SSD is shown in below table:

(0 = segment OFF, 1 = segment ON)

PA7	PA6	PA5	PA4	PA3	PA2	PA1	PA0	PORTA	Number to
Dp	g	f	e	d	c	b	a	Data	be display
0	0	1	1	1	1	1	1	0X3F	0
0	0	0	0	0	1	1	0	0X06	1
0	1	0	1	1	0	1	1	0X5B	2
0	1	0	0	1	1	1	1	0X4F	3
0	1	1	0	0	1	1	0	0X66	4
0	1	1	0	1	1	0	1	0X6D	5
0	1	1	1	1	1	0	1	0X7D	6
0	0	0	0	0	1	1	1	0X07	7
0	1	1	1	1	1	1	1	0X7F	8
0	1	1	0	1	1	1	1	0X6F	9

To select the digit position (out of 4) of the display where data to be display, prefer following table.

(to select the digit position, make that bit equal to 0)

PC3	PC2	PC1	PC0	PORTC	Digit to be
D4	D3	D2	D1	Data	selected
1	1	1	0	0X0E	0
1	1	0	1	0X0D	1
1	0	1	1	0X0B	2
0	1	1	1	0X07	3

Program 1:

```
#include <avr/io.h>
#define F_CPU 4000000UL
#include <util/delay.h>
int main(void)
{
MCUCSR|=(1<<JTD); // sendig 1 to JTD     /*diable JTAG port*/
MCUCSR|=(1<<JTD);
unsigned char ssd_data[10]={0x3f,0x06,0x5b,0x4f,0x66,0x6d,0x7d,0x07,0x7f,0x6f},i;
unsigned char ssd_digit[4]={0x0E,0x0D,0x0B,0x07};
DDRA=0xFF;   // Make PORTA as o/p
DDRC=0xFF;   // Make PORTC as o/p
   while(1)
   {
            PORTC = ssd_digit[0];
            for(i=0;i<10;i++)
            {
                     PORTA = ssd_data[i];
                     _delay_ms(500);
            }
   }
}
```

Program 2:

```
#include <avr/io.h>
#define F_CPU 4000000UL
#include <util/delay.h>
int main(void)
{
         MCUCSR|=(1<<JTD); // sendig 1 to JTD     /*diable JTAG port*/
         MCUCSR|=(1<<JTD);
         unsigned char ssd_data[10]={0x3f,0x06,0x5b,0x4f,0x66,0x6d,0x7d,0x07,0x7f,0x6f},i,j;
         unsigned char ssd_digit[4]={0x0E,0x0D,0x0B,0x07};
         DDRA=0xFF;   // Make PORTA as o/p
         DDRC=0xFF;   // Make PORTC as o/p
         while(1)
```

```
        {
                for(i=0;i<10;i++)
                {
                        for(j=0;j<4;j++)
                        {
                                PORTC = ssd_digit[j];
                                PORTA = ssd_data[i];
                                _delay_ms(250);
                        }
                }
        }
}
```

Program 3:

```
#include <avr/io.h>
#define F_CPU 4000000UL
#include <util/delay.h>
int main()
{
        MCUCSR|=(1<<JTD); // sendig 1 to JTD   /*diable JTAG port*/
        MCUCSR|=(1<<JTD);
        unsigned char ssd[10]={0x3f,0x06,0x5b,0x4f,0x66,0x6d,0x7d,0x07,0x7f,0x6f},i, delay;
        DDRA=0xFF;   // Make PORTA as o/p
        DDRC=0xFF;   // Make PORTC as o/p
        while(1)
        {
                for(i=9;i>0;i--)
                {
                        for(delay=0;delay<20;delay++)
                        {
                                PORTC = 0x07;
                                PORTA=ssd[i];
                                _delay_ms(10);
                                if(i<9)
                                {
                                        PORTC = 0x0B;
```

```
                    PORTA=ssd[i+1];
                    _delay_ms(10);
                    if(i<8)
                    {
                        PORTC = 0x0D;
                        PORTA=ssd[i+2];
                        _delay_ms(10);
                        if(i<7)
                        {
                            PORTC = 0x0E;
                            PORTA=ssd[i+3];
                            _delay_ms(10);
                        }
                    }
                }
            }
        }
    }
}
```

Result: Successfully studied the Interfacing of SSD with AVR microcontroller and observe the output.

Teacher's Sign

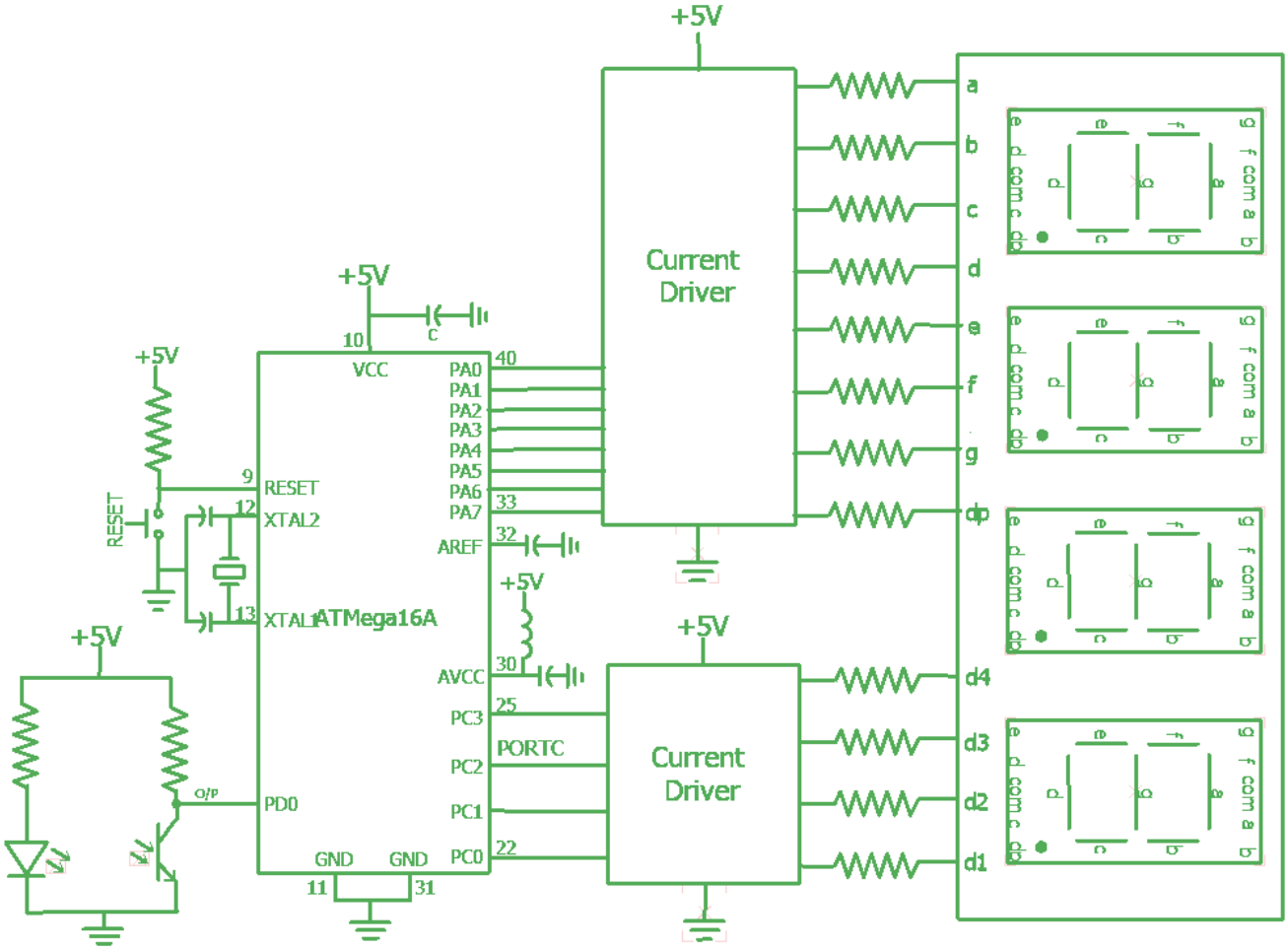
----------*********----------

Experiment – 5

Title: Event Counter using Opto-interrupter

Aim: To Study of Event Counter using opto-interrupter and SSD.

Circuit Diagram:



Theory:

An Opto Interrupter also known as photo interrupter is a transmission-type photo sensor that integrates optical receiving and transmitting elements in a single package. This device will turn a circuit on and off optically. It consists of a molded plastic housing with an IR LED facing a photo transistor across a gap. Any object in the gap will interrupt the IR beam and consequently switch the photo transistor on and off. The device is very fast and ideal for counting, timing or sensing. It is widely used in Optical Switch, ATM Machines, Vending Machines, Edge and Position Detection, Office Automation. The Opto switch or interrupter is a small U shaped black plastic package which has four legs-2 for infra-red LED on one side of the U and 2 for light sensitive transistor on the other. Both are inbuilt, no external connection is required. With both the infrared light source and the photo transistor in a single package, the photo-interrupter employs transmissive technology to sense obstacles existence, acts as on / off switchers or even to sense low resolution rotary or linear motions. i.e The principle states

that objects opaque to infrared will interrupt the transmission of light between an infrared emitting diode and a photo sensor switching the output from an "ON" state to an "OFF" state.

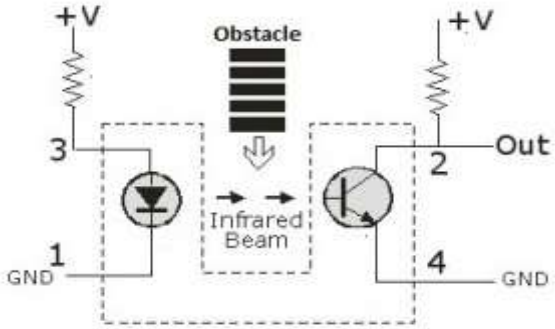


Fig. Operation of Opto-interrupter

Procedure:

- Start AVR Studio software on PC and write a program.
- Connect the USB cable between USB ASP programmer on the Target board and PC.
- Turn ON the power supply of the target board and then load the .hex file into the AVR using "ProgISP" software.
- Turn OFF the power supply while making circuit connections on the board. Make necessary connections as shown in the circuit diagram.
- Switch ON the power supply of the target board and observe the output.

Program:

```
//Opto-interrrupter o/p connected to PD0
#include <avr/io.h>
int main()
{
/*diable JTAG port*/
 MCUCSR|=(1<<JTD); // sendig 1 to JTD
 MCUCSR|=(1<<JTD);

 unsigned char ssd[10]={0x3f,0x06,0x5b,0x4f,0x66,0x6d,0x7d,0x07,0x7f,0x6f}, count=0, sensor;
 DDRA=0xFF;   // Make PORTA as o/p
 DDRC=0xFF;   // Make PORTC as o/p
 DDRD=0x00;   // Make PORTD as i/p
 PORTC = 0x0E;
 PORTA = ssd[count];
```

```
while(1)
{
sensor = PIND;
sensor = sensor & 0x01;
if (sensor == 0x01)
{
        count++;
        if(count>9)
        {
                count = 0;
        }
        PORTC = 0x0E;
        PORTA = ssd[count];
        while(sensor == 0x01)
        {
                sensor = PIND;
                sensor = sensor & 0x01;
        }
}
}
}
```

Result: Interfacing of Opto interrupter as an event counter with AVR is successfully studied, tested and observe the output.

Teacher's Sign.

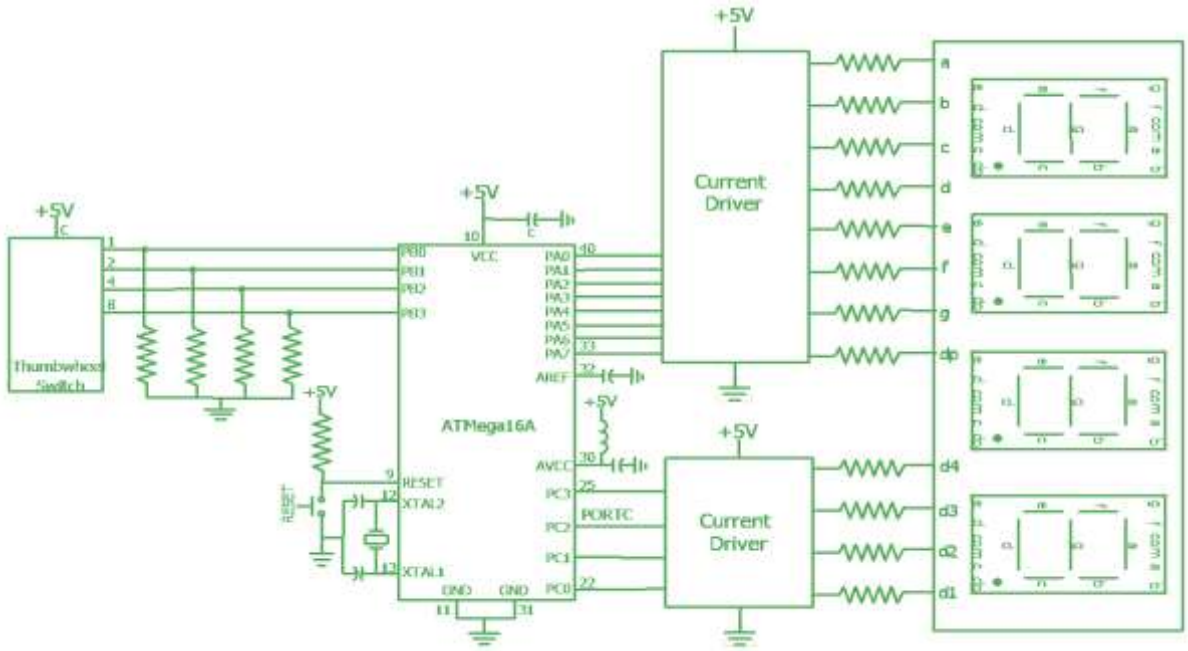
----------**********----------

Experiment – 6

Title: Study of Interfacing Thumb Wheel Switch & display input data on SSD

Aim: To study interfacing of thumb wheel switch to AVR and display input data on SSD.

Circuit Diagram:



Theory:

Thumbwheel Switch: Digital computers, microprocessors, and other digital systems are binary operated. The data is processed in the form of '0's and '1's. However, we communicate in the form of decimal numbers and alphabetic characters only. Therefore, the need arias for interfacing between digital systems and human operators. To demonstrate this task, we would use Thumbwheel switch as input device, and seven-segment display as output device. The process of converting familiar code to is known as encoding. The reverse process of converting binary to familiar code known as decoding.

Numbers can be represented in large variety of codes. Computers and other digital circuits are required to handle data, which may be numerics, alphabets or special characters are to be converted into binary form. For convenience binary code is always grouped into three bit groups (octal code), four bit group (BCD or Hexadecimal code). Unfortunately, different number systems are not much popular except decimal number system. Decimal number is most popular today for setting time angle, counts in digit system. Thumb wheel switch converts decimal number into BCD form, which could be further processed by the systems and result is displayed once again in decimal form. We can provide decimal number using thumbwheel switch.

Procedure:

- Start AVR Studio software on PC and write a program.

- Connect the USB cable between USB ASP programmer on the Target board and PC.
- Turn ON the power supply of the target board and then load the .hex file into the AVR using "ProgISP" software.
- Turn OFF the power supply while making circuit connections on the board. Make necessary connections as shown in the circuit diagram.
- Switch ON the power supply of the target board and observe the output.

Program:

```
#include <avr/io.h>
#define F_CPU 4000000UL
#include <util/delay.h>
int main()
{
 MCUCSR|=(1<<JTD);      /*diable JTAG port*/
 MCUCSR|=(1<<JTD);
 unsigned char ssd[10]={0x3f,0x06,0x5b,0x4f,0x66,0x6d,0x7d,0x07,0x7f,0x6f}, sw=0;
 DDRA=0xFF;   // Make PORTA as o/p
 DDRC=0xFF;   // Make PORTC as o/p
 DDRB=0x00;   // Make PORTB as i/p
 while(1)
 {
        sw = PINB;
        sw = sw & 0x0F;
        PORTC = 0x0E;
        PORTA = ssd[sw];
        _delay_ms(300);
        _delay_ms(300);
 }
}
```

Result: Interfacing of Thumbwheel Switch with AVR is studied & observe input data on SSD.

Teacher's Sign.

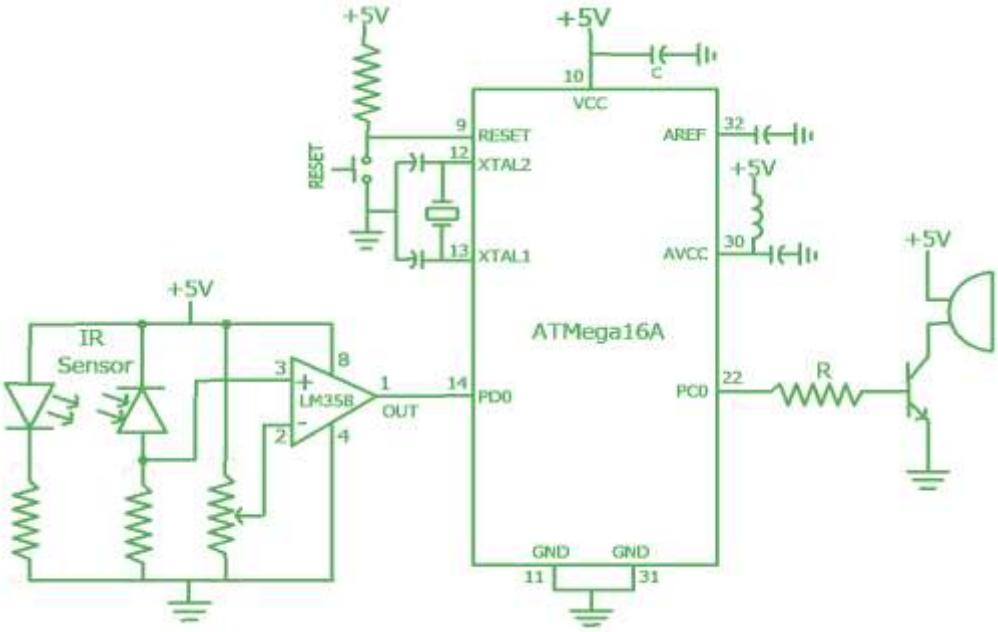
----------**********----------

Experiment – 7

Title: Study of Intrusion detection security system using IR sensor and Buzzer

Aim: To study Intrusion detection security system using IR sensor and Buzzer.

Circuit Diagram:



TECHSOLUTION
NEW

Theory:

An intrusion detection system is a security system, which is a group of physical electronic components that all work together to protect us from any type of intrusion (suspicious activity) in a home or office or store. In this practical we use IR sensor and buzzer to make this application using AVR microcontroller. An infrared proximity sensor or IR Sensor is an electronic device that emits infrared lights to sense some aspect of the surroundings and can be employed to detect the motion of an object. As this is a passive sensor, it can only measure infrared radiation. This sensor is very common in the electronic industry and if you've ever tried to design an obstacle avoidance robot or any other proximity detection-based system, chances are you already know about this sensor.

Out pin is a 5V TTL logic output. LOW indicates no motion is detected; HIGH means motion is detected. The working of the IR sensor module is very simple, it consists of two main components: the first is the IR transmitter section and the second is the IR receiver section. In the transmitter section, IR led is used and in the receiver section, a photodiode is used to receive infrared signal and after some signal processing and conditioning, you will get the output. An IR proximity sensor works by applying a voltage to the onboard Infrared Light Emitting Diode which in turn emits infrared light. This light propagates through the air and hits an object, after that the light gets reflected in the photodiode sensor. If the object is close, the reflected light will be stronger, if the object is far away, the reflected light will be weaker.

Procedure:

- Start AVR Studio software on PC and write a program.
- Connect the USB cable between USB ASP programmer on the Target board and PC.
- Turn ON the power supply of the target board and then load the .hex file into the AVR using "ProgISP" software.
- Turn OFF the power supply while making circuit connections on the board. Make necessary connections as shown in the circuit diagram.
- Switch ON the power supply of the target board and observe the output.

Program:

```
#include <avr/io.h>
int main()
{
char sensor = 0;
DDRC=0xFF;   // Make PORTC as o/p
DDRD=0x00;   // Make PORTD as i/p
while(1)
{
sensor = PIND;
sensor = sensor & 0x01;
if (sensor == 0x00)
{
PORTC = 0x01;    //make buzzer ON
}
else
{
PORTC = 0x00;     //make buzzer OFF
}
}
}
```

Result: Interfacing of IR sensor & buzzer with AVR microcontroller for Intrusion detection security system is studied successfully.

Teacher's Sign.

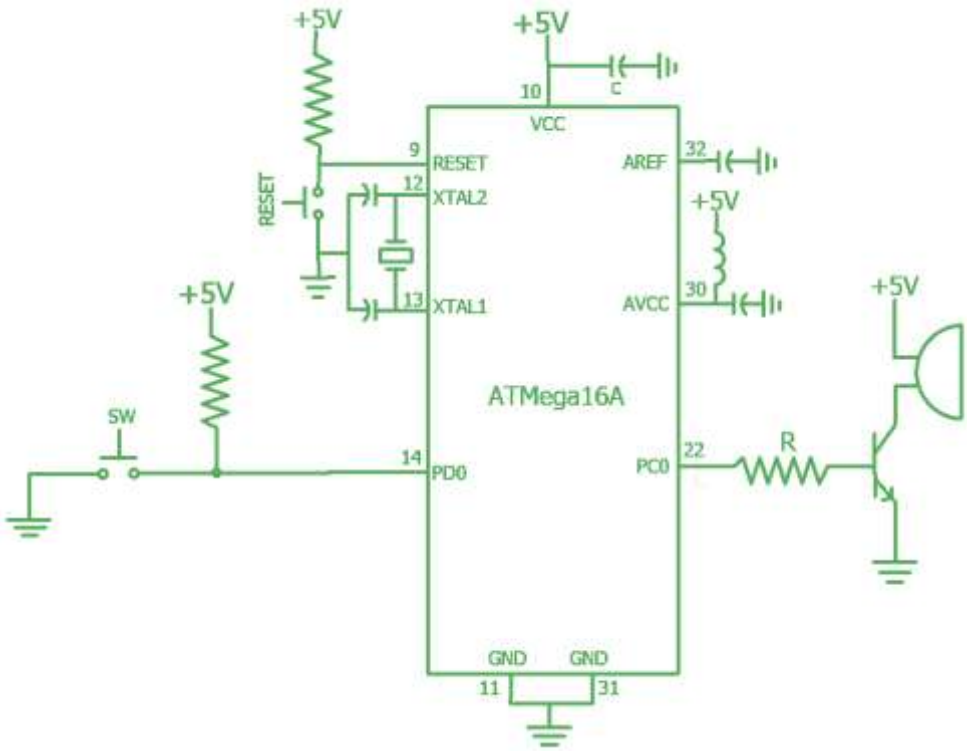
----------**********----------

Experiment – 8

Title: Get the status of the switch and turn on / off LED / buzzer

Aim: To write a program to get the status of the switch and turn on / off buzzer.

Circuit Diagram:



Theory:

In electronics, a switch is a fundamental component that controls the flow of electrical current within a circuit, acting as a gate to either allow or prevent the passage of electricity. Essentially, it's a device that can be toggled to create an open or closed circuit, thus turning a device or part of a circuit "on" or "off". A switch interrupts or diverts the flow of current in a circuit, effectively acting as a conductor when closed and an insulator when open.

A buzzer is a simple electronic device that produces an audible sound, often a buzzing or beeping noise, when activated by an electrical signal. It's commonly used as an audio signaling device in various applications like alarms, timers, and feedback mechanisms in electronic devices. Buzzers can be broadly classified into two main types: piezoelectric and electromagnetic.

Procedure:

- Start AVR Studio software on PC and write a program.
- Connect the USB cable between USB ASP programmer on the Target board and PC.
- Turn ON the power supply of the target board and then load the .hex file into the AVR using "ProgISP" software.

- Turn OFF the power supply while making circuit connections on the board. Make necessary connections as shown in the circuit diagram.
- Switch ON the power supply of the target board and observe the output.

Program:

(Note: As per our circuit diagram Switch or push button output is 1 when not pressed and become 0 when switch is pressed.)

```
#include <avr/io.h>
int main()
{
char sw = 0;
DDRC=0xFF;   // Make PORTC as o/p
DDRD=0x00;   // Make PORTD as i/p
while(1)
{
        sw = PIND;
        sw = sw & 0x01;
        if (sw == 0x00)
        {
          PORTC = 0x01; //make buzzer ON
        }
        else
        {
          PORTC = 0x00; //make buzzer OFF
        }
}
}
```

Result: Successfully studied the interfacing of switch with AVR microcontroller and observe the output.

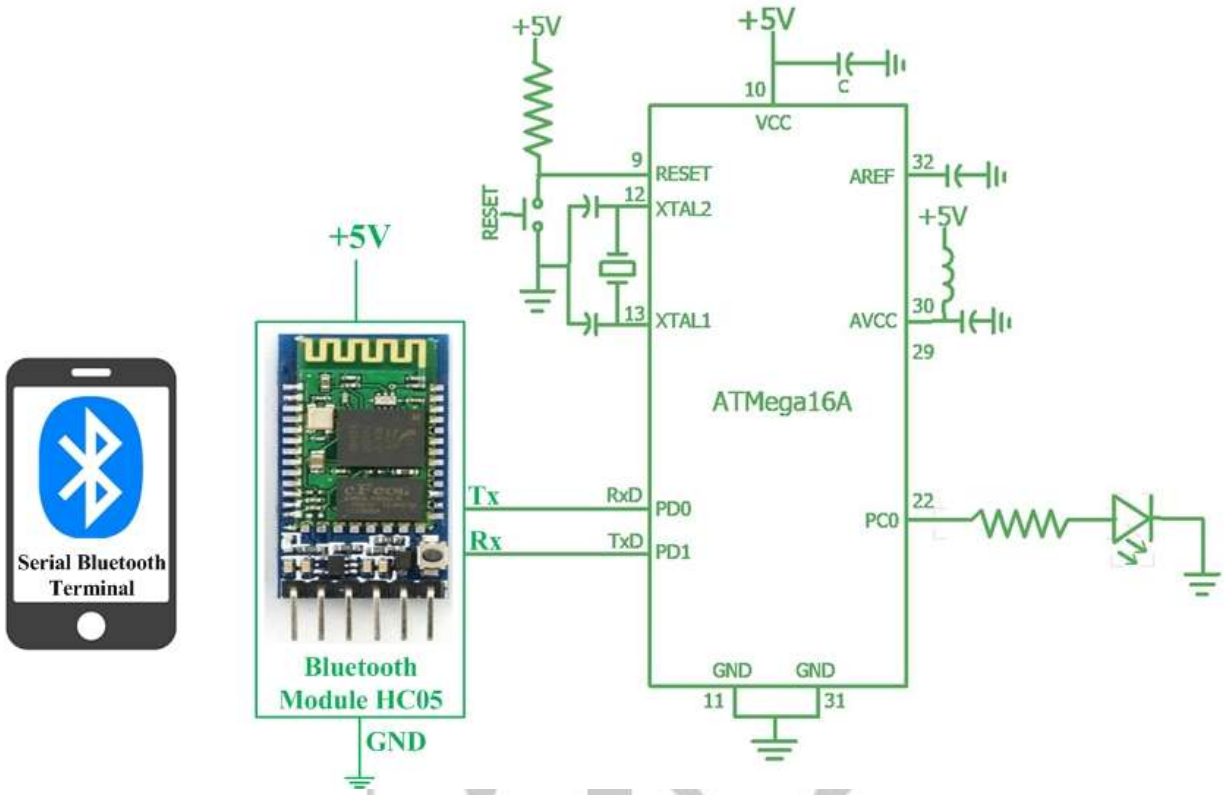
Teacher's Sign

Experiment – 9

Title: Study to Control devices using Smart phone & Bluetooth Module HC05.

Aim: To study device control using Smart phone & Bluetooth Module HC05

Circuit Diagram:



Theory:

Bluetooth is a wireless technology standard used for exchanging data between fixed and mobile devices over short distances using short-wavelength UHF radio waves in the industrial, scientific and medical radio bands, from 2.402 GHz to 2.480 GHz, and building personal area networks (PANs). It was originally conceived as a wireless alternative to RS-232 data cables. The IEEE standardized Bluetooth as IEEE 802.15.1. Bluetooth is a one of the great example for wireless connectivity. It is used in many fields. Bluetooth consumes very small amount of energy. Here we are going to interface a Bluetooth Module (HC-05) with Arduino. HC-05 is a Bluetooth module which can communicate in two ways. Which means, It is full-duplex. We can use it with most micro controllers. Because it operates on Serial Port Protocol (SSP). The module communicates with the help of USART (Universal Synchronous/Asynchronous Receiver/Transmitter) at the baud rate of 9600, and it also supports other baud rate. So we can interface this module with any microcontroller which supports USART. The HC-05 can operate in two modes. One is Data mode and other is AT command mode. When the enable pin is "LOW" the HC-05 is in Data Mode. If that pin set as "HIGH" the module is in AT command mode. Here we operate this module in Data Mode.

Technical Specifications -

- Operating Voltage: 4V to 6V (Typically +5V)
- Operating Current: 30mA
- Range: <100m
- Works with Serial communication (USART) and TTL compatible.
- Can be easily interfaced with Laptop or Mobile phones with Bluetooth.

USART Registers:

USART I/O Data Register (UDR) – The USART Transmit Data Buffer Register and USART Receive Data Buffer Registers share the same I/O address referred to as USART Data Register or UDR. The Transmit Data Buffer Register (TXB) will be the destination for data written to the UDR Register location. Reading the UDR Register location will return the contents of the Receive Data Buffer Register (RXB).

USART Control and Status Register A (UCSRA) –

7	6	5	4	3	2	1	0
RXC	TXC	UDRE	FE	DOR	PE	U2X	MPCM

- **Bit 7 – RXC:** USART Receive Complete

This flag bit is set when there is unread data in UDR. The RXC Flag can be used to generate a Receive Complete interrupt.

- **Bit 6 – TXC:** USART Transmit Complete

This flag bit is set when the entire frame from Tx Buffer is shifted out and there is no new data currently present in the transmit buffer (UDR). The TXC Flag bit is automatically cleared when a transmit complete interrupt is executed, or it can be cleared by writing a one to its bit location. The TXC Flag can generate a Transmit Complete interrupt.

- **Bit 5 – UDRE:** USART Data Register Empty

If UDRE is one, the buffer is empty which indicates the transmit buffer (UDR) is ready to receive new data. The UDRE Flag can generate a Data Register Empty Interrupt. UDRE is set after a reset to indicate that the transmitter is ready.

- **Bit 4 – FE:** Frame Error
- **Bit 3 – DOR:** Data OverRun

This bit is set if a Data OverRun condition is detected. A Data OverRun occurs when the receive buffer is full (two characters) and a new character is waiting in the receive Shift Register.

- **Bit 2 – PE:** Parity Error
- **Bit 1 – U2X:** Double the USART Transmission Speed
- **Bit 0 – MPCM:** Multi-processor Communication Mode

USART Control and Status Register B (UCSRB) –

7	6	5	4	3	2	1	0
RXCIE	TXCIE	UDRIE	RXEN	TXEN	UCSZ2	RXB8	TXB8

- **Bit 7 – RXCIE:** RX Complete Interrupt Enable

  Writing one to this bit enables interrupt on the RXC Flag.

- **Bit 6 – TXCIE:** TX Complete Interrupt Enable

  Writing one to this bit enables interrupt on the TXC Flag.

- **Bit 5 – UDRIE:** USART Data Register Empty Interrupt Enable

  Writing one to this bit enables interrupt on the UDRE Flag.

- **Bit 4 – RXEN:** Receiver Enable

  Writing one to this bit enables the USART Receiver.

- **Bit 3 – TXEN:** Transmitter Enable

  Writing one to this bit enables the USART Transmitter.

- **Bit 2 – UCSZ2:** Character Size

The UCSZ2 bits combined with the UCSZ1:0 bit in UCSRC sets the number of data bits (Character Size) in a frame the receiver and transmitter use.

- **Bit 1 – RXB8:** Receive Data Bit 8
- **Bit 0 – TXB8:** Transmit Data Bit 8

USART Control and Status Register C (UCSRC) –

7	6	5	4	3	2	1	0
URSEL	UMSEL	UPM1	UPM0	USBS	USCZ1	USCZ0	UCPOL

- **Bit 7 – URSEL:** Register Select

This bit selects between accessing the **UCSRC** or the **UBRRH** Register, as both register shares the same address. The URSEL must be one when writing the UCSRC or else data will be written in the UBRRH register.

- **Bit 6 – UMSEL:** USART Mode Select

  This bit selects between the Asynchronous and Synchronous mode of operation.

  0 = Asynchronous Operation

  1 = Synchronous Operation

- **Bit 5:4 – UPM1:0**: Parity Mode

These bits enable and set the type of parity generation and check. If parity a mismatch is detected, the PE Flag in UCSRA will be set.

UPM1	UPM0	Parity Mode
0	0	Disabled
0	1	Reserved
1	0	Enabled, Even Parity
1	1	Enabled, Odd Parity

- **Bit 3 – USBS**: Stop Bit Select

This bit selects the number of Stop Bits to be inserted by the Transmitter. The Receiver ignores this setting.

0 = 1-bit

1 = 2-bit

- **Bit 2:1 – UCSZ1:0**: Character Size

The UCSZ1:0 bits combined with the UCSZ2 bit in UCSRB sets the number of data bits (Character Size) in a frame the Receiver and Transmitter use.

UCSZ2	UCSZ1	UCSZ0	Character Size
0	0	0	5-bit
0	0	1	6-bit
0	1	0	7-bit
0	**1**	**1**	**8-bit**
1	0	0	Reserved
1	0	1	Reserved
1	1	0	Reserved
1	1	1	9-bit

- **Bit 0 – UCPOL:** Clock Polarity

This bit is used for synchronous mode only. Write this bit to zero when the asynchronous mode is used.

USART Baud Rate Registers (UBRRL and UBRRH) –

UBRRL and UBRRH: USART Baud Rate Registers

15	14	13	12	11	10	9	8	
URSEL	–	–	–	UBRR[11:8]				UBRRH
UBRR[7:0]								UBRRL
7	6	5	4	3	2	1	0	

- **Bit 15 – URSEL:** Register Select

This bit selects between accessing the **UCSRC** or the **UBRRH** Register, as both register shares the same address. The URSEL must be one when writing the UCSRC or else data will be written in the UBRRH register.

- **Bit 11:0 – UBRR11:0:** USART Baud Rate Register.

Used to define the baud rate

Formula for Baud Rate = Fosc / (16 * (UBBR + 1)). If want know the baud rate and want to find value of UBBR, then formula becomes,

UBBR = (Fosc / (16 * Baudrate)) – 1

UBRR = (4MHz / (16*9600)) – 1 = 25

Procedure:

- Start AVR Studio software on PC and write a program.
- Connect the USB cable between USB ASP programmer on the Target board and PC.
- Turn ON the power supply of the target board and then load the .hex file into the AVR using "ProgISP" software.
- Turn OFF the power supply while making circuit connections on the board. Make necessary connections as shown in the circuit diagram.
- Switch ON the power supply of the target board and observe the output.

Program:

```
#include <avr/io.h>
#define F_CPU 4000000UL                /* Define frequency here its 4MHz */
#include <util/delay.h>

void UART_init()
{
UBRRH =  0;            /* Load upper 8-bits*/
UBRRL = 25;            /* Load lower 8-bits of the baud rate value */
UCSRC = (1<<URSEL) | (1<<UCSZ0) | (1<<UCSZ1);        /* Use 8-bit character sizes */
UCSRB = (1 << RXEN) | (1 << TXEN);                   /* Turn on transmission and reception */
}

char UART_RxChar()
{
while ((UCSRA & (1 << RXC)) == 0);       /* Wait till data is received */
return(UDR);                             /* Return the byte*/
}

int main()
{
char cmd=0;
DDRC = 0xFF;      // Set Port A as output
UART_init();
while(1)
{
cmd = UART_RxChar();
```

```
if(cmd=='1')            //To ON the device send ASCII value of 1
{
PORTC = 0xFF;       // Devices ON
}
else if(cmd=='2')    //To OFF the device send ASCII value of 2
{
PORTC = 0x00;       // Devices OFF
}
}
}
```

TECHSOLUTION
NEW

Result: Interfacing of Bluetooth module with AVR microcontroller to control different devices using Smartphone is studied successfully and observed the o/p.

Teacher's Sign

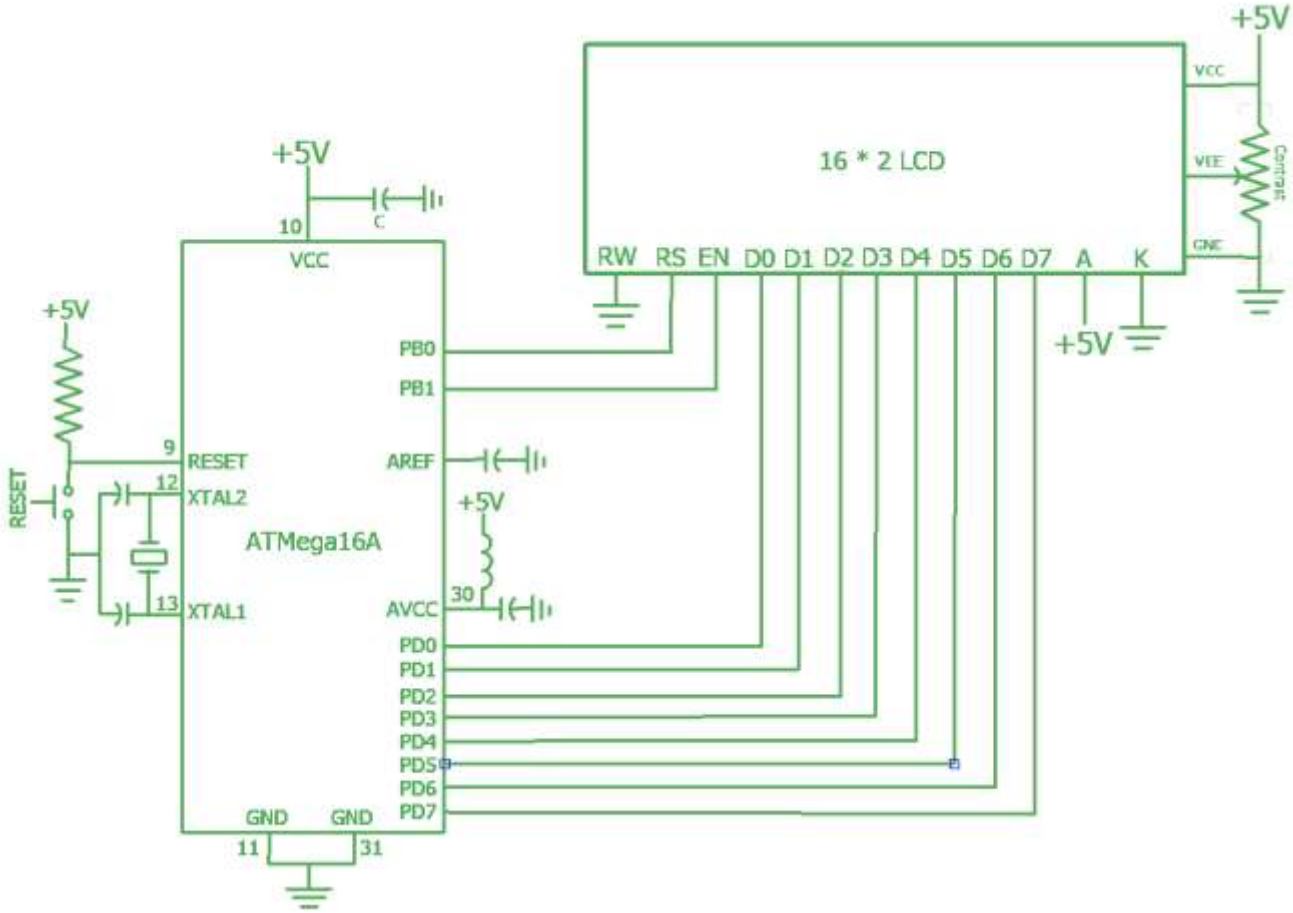
----------**********----------

Experiment – 10

Title: Study of 16*2 LCD interfacing and its programming

Aim: To interface 16*2 LCD display to AVR and display message on LCD.

Circuit Diagram:



Theory:

LCD: A liquid crystal display (LCD) is a thin, flat electronic visual display that uses the light modulating properties of liquid crystals (LCs). They are used in a wide range of applications including: computer monitors, television, instrument panels, aircraft, cockpit display, signage, etc. It is an electronically modulated device made up of any number of pixels filled with liquid crystals. LCD has become very popular option for displaying in Embedded Applications. Since they are very cheap and easy to interface with microcontrollers, they are widely found in devices like telephones, vending machines, washing machines, toys etc. LCD comes in several varieties i.e. 16*2, 20*2, 20*4 etc. These different LCD varieties can display different number of characters i.e. 16*2 can display 32 characters at a time. The 16*2 model has 2 lines and 16 columns of display blocks. Each block can be used to display 1 character. So there are total 32 such blocks. One block has 8*5 pixels. Depending on which pixel is ON and which is OFF we can display several Alpha-Numeric characters. LCD also has a backlight,

which helps us to see the display even in dark. In reality this module consists of a controller chip, a segment driver chip, LCD display and some passive components. There are total 16 pins in the LCD module. While using LCD, we can think a simple analogy for its operation. Each of the 32 blocks is a memory, as soon as we write an ASCII number into one of these 32 memory locations the corresponding character is displayed on that block. The function of displaying the character after decoding the data is done by an onboard controller chip. The following table shows the LCD pin diagram, LCD commands.

INTERFACING LCD TO 8051

LCD Pin Descriptions

Pin Descriptions for LCD

Pin	Symbol	I/O	Descriptions
1	VSS	--	Ground
2	VCC	--	+5V power supply
3	VEE	--	Power supply to control contrast
4	RS	I	RS=0 to select command register, RS=1 to select data register
5	R/W	I	R/W=0 for write, R/W=1 for read
6	E	I/O	Enable
7	DB0	I/O	The 8-bit data bus
8	DB1	I/O	The 8-bit data bus
9	DB2	I/O	The 8-bit data bus
10	DB3	I/O	The 8-bit data bus
11	DB4	I/O	The 8-bit data bus
12	DB5	I/O	The 8-bit data bus
13	DB6	I/O	The 8-bit data bus
14	DB7	I/O	The 8-bit data bus

- Send displayed information or instruction command codes to the LCD
- Read the contents of the LCD's internal registers

used by the LCD to latch information presented to its data bus

Figure: LCD pin diagram

COMMAND	COMMAND CODE										COMMAND CODE	E-CYCLE f_{osc}=250KHz
	RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0		
SCREEN CLEAR	0	0	0	0	0	0	0	0	0	1	Screen Clear, Set AC to 0 Cursor Reposition	1.64ms
CURSOR RETURN	0	0	0	0	0	0	0	0	1	*	DDRAM AD=0, Return, Content Changeless	1.64ms
INPUT SET	0	0	0	0	0	0	0	1	I/D	S	Set moving direction of cursor, Appoint if move	40us
DISPLAY SWITCH	0	0	0	0	0	0	1	D	C	B	Set display on/off,cursor on/off, blink on/off	40us
SHIFT	0	0	0	0	0	1	S/C	R/L	*	*	Remove cursor and whole display,DDRAM changeless	40us
FUNCTION SET	0	0	0	0	1	DL	N	F	*	*	Set DL,display line,font	40us
CGRAM AD SET	0	0	0	1	ACG						Set CGRAM AD, send receive data	40us
DDRAM AD SET	0	0	1	ADD							Set DDRAM AD, send receive data	40us
BUSY/AD READ CT	0	1	BF	AC							Executing internal function, reading AD of CT	40us
CGRAM/ DDRAM DATA WRITE	1	0	DATA WRITE								Write data from CGRAM or DDRAM	40us
CGRAM/ DDRAM DATA READ	1	1	DATA READ								Read data from CGRAM or DDRAM	40us
	I/D=1: Increment Mode; I/D=0: Decrement Mode S=1: Shift S/C=1: Display Shift; S/C=0: Cursor Shift R/L=1: Right Shift; R/L=0: Left Shift DL=1: 8D DL=0: 4D N=1: 2R N=0: 1R F=1: 5x10 Style; F=0: 5x7 Style BF=1: Execute Internal Function; BF=0: Command Received										DDRAM: Display data RAM CGRAM: Character Generator RAM ACG: CGRAM AD ADD: DDRAM AD & Cursor AD AC: Address counter for DDRAM & CGRAM	E-cycle changing with main frequency. Example: If fcp or f_{osc}=270KHz 40us x 250/270 =37us

Figure: LCD commands

Procedure:

- Start AVR Studio software on PC and write a program.
- Connect the USB cable between USB ASP programmer on the Target board and PC.
- Turn ON the power supply of the target board and then load the .hex file into the AVR using "ProgISP" software.
- Turn OFF the power supply while making circuit connections on the board. Make necessary connections as shown in the circuit diagram.
- Switch ON the power supply of the target board and observe the output.

Program:

```
#include <avr/io.h>
#define F_CPU 4000000UL
#include <util/delay.h>
void lcd_init();
void lcd_cmd(unsigned char value);
void lcd_data(unsigned char value);
void lcd_print(char *str);

int main(void)
{
lcd_init();
lcd_print("MGTechSolution");
}

void lcd_init()
{
DDRB = 0xFF;    //make PORTB as o/p
DDRD = 0xFF;    //make PORTD as o/p
_delay_ms(50);
lcd_cmd(0x38);
lcd_cmd(0x0C);
lcd_cmd(0x06);
lcd_cmd(0x01);
lcd_cmd(0x80);
}
```

```
void lcd_cmd(unsigned char value)
{
PORTD = value;
PORTB &= 0b11111110;    //RS = 0
PORTB|=0b00000010;      //EN = 1
_delay_us(2);
PORTB &= 0b11111101;    //EN = 0
_delay_ms(3);

void lcd_data(unsigned char value)
{
PORTD = value;
PORTB|=0b00000001;      //RS = 1
PORTB|=0b00000010;      //EN = 1
_delay_us(2);
PORTB &= 0b11111101;    //EN = 0
_delay_ms(3);
}

void lcd_print(char *str)
{
unsigned char i;
for (i = 0; str[i] != '\0'; i++)
{
lcd_data(str[i]);
_delay_ms(3);
}
}
```

Result: Interfacing of 16*2 LCD with AVR microcontroller is studied successfully and observed the message displayed on LCD.

Teacher's Sign

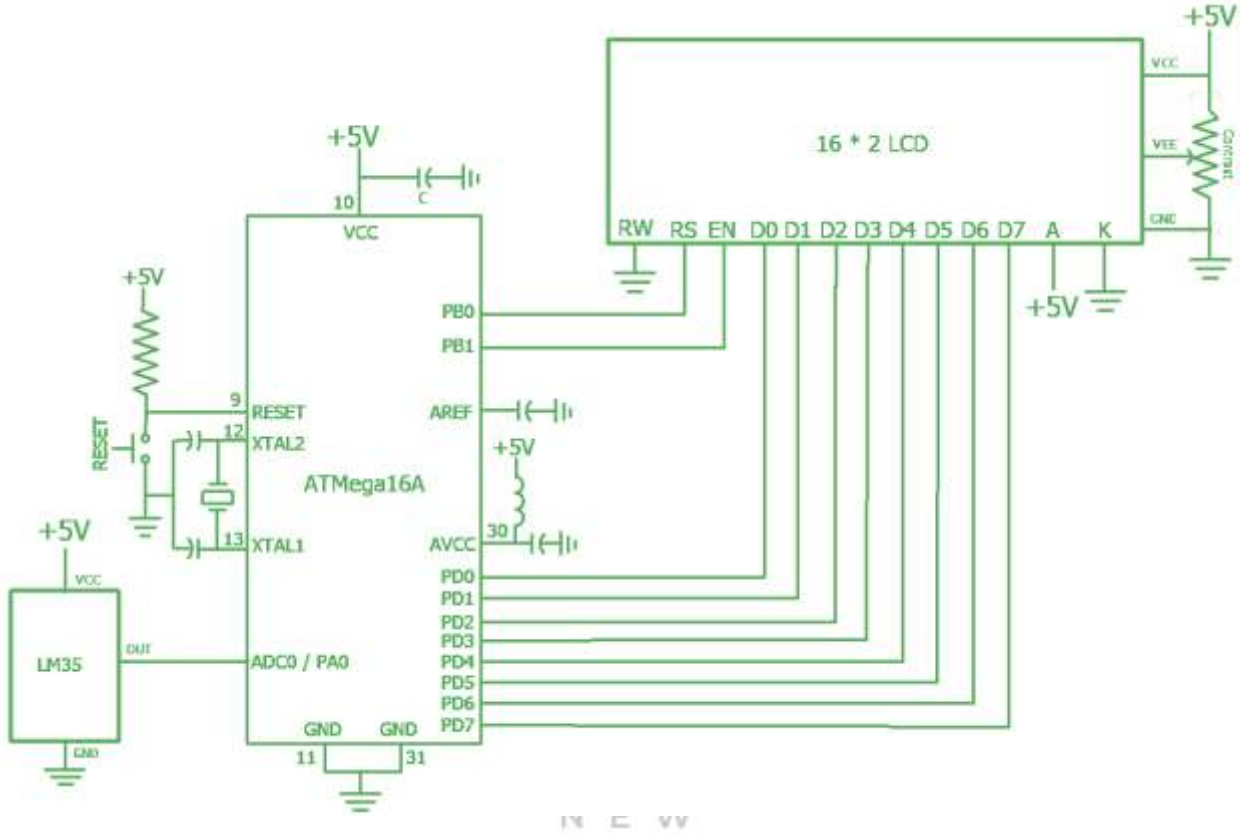
----------**********----------

Experiment – 11

Title: Study of interfacing LM35 sensor to on chip ADC of AVR and display temp on LCD

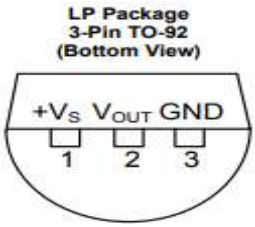
Aim: To study interfacing of LM35 sensor to on chip ADC of AVR and display temp on LCD.

Circuit Diagram:



Theory:

The LM35 series are precision integrated-circuit temperature devices with an output voltage linearly proportional to the Centigrade temperature. The LM35 device does not require any external calibration or trimming to provide typical accuracies of ±¼°C at room temperature and ±¾°C over a full −55°C to 150°C temperature range. Lower cost is assured by trimming and calibration at the wafer level. The low-output impedance, linear output, and precise inherent calibration of the LM35 device makes interfacing to readout or control circuitry especially easy. This sensor has sensitivity of 10mv/°C, operates from 4 V to 30 V.



Pin Diagram of LM 35 Sensor

ATmega16 ADC Features:

- It is 10-bit ADC
- Converted output binary data is held in two special functions 8-bit register ADCL (result Low) and ADCH (result in High).
- ADC gives 10-bit output, so (ADCH: ADCL) only 10-bits are useful out of 16-bits.
- We have options to use this 10-bits as upper bits or lower bits.
- We also have three options for Vref. 1. AVcc (analog Vcc), 2. Internal 2.56 v3. External Aref. Pin.
- The total conversion time depends on crystal frequency and ADPS0: 2 (frequency devisor)

ATmega16 ADC Registers:

In AVR ADC, we need to understand four main register -

1. ADCH: Holds digital converted data higher byte
2. ADCL: Holds digital converted data lower byte
3. ADMUX: ADC Multiplexer selection register
4. ADCSRA: ADC Control and status register

ADMUX: ADC Multiplexer selection register –

7	6	5	4	3	2	1	0
REFS1	REFS0	ADLAR	MUX4	MUX3	MUX2	MUX1	MUX0

Bit 7: 6 – REFS1 : 0: Reference Selection Bits

Reference voltage selection for ADC

REFS1	REFS0	Vref to ADC
0	0	AREF pin
0	1	AVCC pin i.e. Vcc 5 V
1	0	Reserved
1	1	Internal 2

Bit 5 – ADLAR: ADC Left Adjust Result

Use 10-bits output as upper bits or lower bits in ADCH & ADCL.

	ADCH								ADCL		
Left-Justified **ADLAR = 1**	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0	UNUSED
ADLAR = 0 Right-Justified	UNUSED						D9	D8	D7	D6	D5	D4	D3	D2	D1	D0

Bits 4 : 0 – MUX4 : 0: Analog Channel and Gain Selection Bits

We can select input channel ADC0 to ADC7 by using these bits. These bits are also used to select comparator (inbuilt in AVR) inputs with various gain. We will cover these comparator operations in another part.

Selecting a channel is very easy, just put the channel number in MUX4 : 0.

Suppose you are connecting the input to ADC channel 2 then put 00010 in MUX4 : 0.

Suppose you are connecting the input to ADC channel 5 then put 00101 in MUX4 : 0.

ADCSRA: ADC Control and status register -

7	6	5	4	3	2	1	0
ADEN	ADSC	ADATE	ADIF	ADIE	ADPS2	ADPS1	ADPS0

- **Bit 7 – ADEN: ADC Enable**

Writing one to this bit enables the ADC. By writing it to zero, the ADC is turned off. Turning the ADC off while a conversion is in progress, will terminate this conversion.

- **Bit 6 – ADSC: ADC Start Conversion**

Writing one to this bit starts the conversion.

- **Bit 5 – ADATE: ADC Auto Trigger Enable**

Writing one to this bit, results in Auto Triggering of the ADC is enabled.

- **Bit 4 – ADIF: ADC Interrupt Flag**

This bit is set when an ADC conversion completes and the Data Registers are updated.

- **Bit 3 – ADIE: ADC Interrupt Enable**

Writing one to this bit, the ADC Conversion Complete Interrupt is activated.

- **Bits 2 : 0 – ADPS2 : 0: ADC Prescaler Select Bits**

These bits determine the division factor between the XTAL frequency and the input clock to the ADC

ADPS2	ADPS1	ADPS0	Division Factor
0	0	0	2
0	0	1	2
0	1	0	4
0	1	1	8
1	0	0	16
1	0	1	32
1	1	0	64
1	1	1	128

Steps to Program ATmega16 ADC:

- Make the ADC channel pin as an input.
- Set ADC enable bit in ADCSRA, select the conversion speed using ADPS2 : 0. For example, we will select devisor 128.
- Select ADC reference voltage using REFS1: REFS0 in ADMUX register, for example, we will use AVcc as a reference voltage.
- Select the ADC input channel using MUX4 : 0 in ADMUX, for example, we will use channel 0.
- Start conversion by setting bit ADSC in ADCSRA.
- Wait for conversion to complete by polling ADIF bit in ADCSRA register.
- After the ADIF bit gone high, read ADCL and ADCH register to get digital output.
- Notice that read ADCL before ADCH; otherwise result will not be valid.

Procedure:

- Start AVR Studio software on PC and write a program.
- Connect the USB cable between USB ASP programmer on the Target board and PC.
- Turn ON the power supply of the target board and then load the .hex file into the AVR using "ProgISP" software.
- Turn OFF the power supply while making circuit connections on the board. Make necessary connections as shown in the circuit diagram.
- Switch ON the power supply of the target board and observe the output.

TECHSOLUTION
NEW

Program:

```
#include <avr/io.h>
#define F_CPU 4000000UL
#include <util/delay.h>
#include <stdlib.h> // Required for itoa()

void adc_init();
int read_adc(char channel);
void lcd_init();
void lcd_cmd(unsigned char value);
void lcd_data(unsigned char value);
void lcd_print(char *str);
```

```
int main() {
unsigned int adc_value;
unsigned char temp;
char display_buffer[6];
// Initialize ADC & LCD
adc_init();
lcd_init();
while (1) {
// Read ADC channel 0
adc_value = read_adc(0);
adc_value = adc_value * 5;  //resolution of ADC is 4.88mv
temp = adc_value / 10;     //LM35 sensitivity 10mV/C
lcd_cmd(0x01);
lcd_print("Temp= ");
itoa(temp, display_buffer, 10); // Convert to base 10
lcd_print(display_buffer);
// Delay for some time
_delay_ms(500);
}
}

void adc_init() {
DDRA=0x00;  /* Make ADC port as input */
// Set ADC reference voltage to AVCC
ADMUX |= (1 << REFS0);
// Set prescaler to 128
ADCSRA |= (1 << ADPS2) | (1 << ADPS1) | (1 << ADPS0);
// Enable ADC
ADCSRA |= (1 << ADEN);
}

int read_adc(char channel) {
// Select ADC channel
ADMUX = (ADMUX & 0xF0) | (channel & 0x0F);
// Start conversion
```

NEW MG TECHSOLUTION

```
ADCSRA |= (1 << ADSC);
while (!(ADCSRA & (1<<ADIF)));  /* Wait until end of conversion by polling ADC interrupt flag */
ADCSRA |= (1<<ADIF);            /* Clear interrupt flag */
_delay_ms(1);                   /* Wait a little bit */
return ADCW;                    /* Return ADC word */
}
void lcd_init()
{
DDRB = 0xFF;    //make PORTB as o/p
DDRD = 0xFF;    //make PORTD as o/p
_delay_ms(50);
lcd_cmd(0x38);
lcd_cmd(0x0C);
lcd_cmd(0x06);
lcd_cmd(0x01);
lcd_cmd(0x80);
}

void lcd_cmd(unsigned char value)
{
PORTD = value;
PORTB &= 0b11111110;    //RS = 0
PORTB|=0b00000010;      //EN = 1
_delay_us(2);
PORTB &= 0b11111101;    //EN = 0
_delay_ms(3);
}
void lcd_data(unsigned char value)
{
PORTD = value;
PORTB|=0b00000001;      //RS = 1
PORTB|=0b00000010;      //EN = 1
_delay_us(2);
PORTB &= 0b11111101;    //EN = 0
_delay_ms(3);
```

TECHSOLUTION NEW

```
}

void lcd_print(char *str)
{
unsigned char i;
for (i = 0; str[i] != '\0'; i++)
{
lcd_data(str[i]);
_delay_ms(3);
}
}
```

TECHSOLUTION
NEW

Result: Interfacing of LM35 sensor with AVR microcontroller is studied successfully and observed the temperature on LCD.

Teacher's Sign

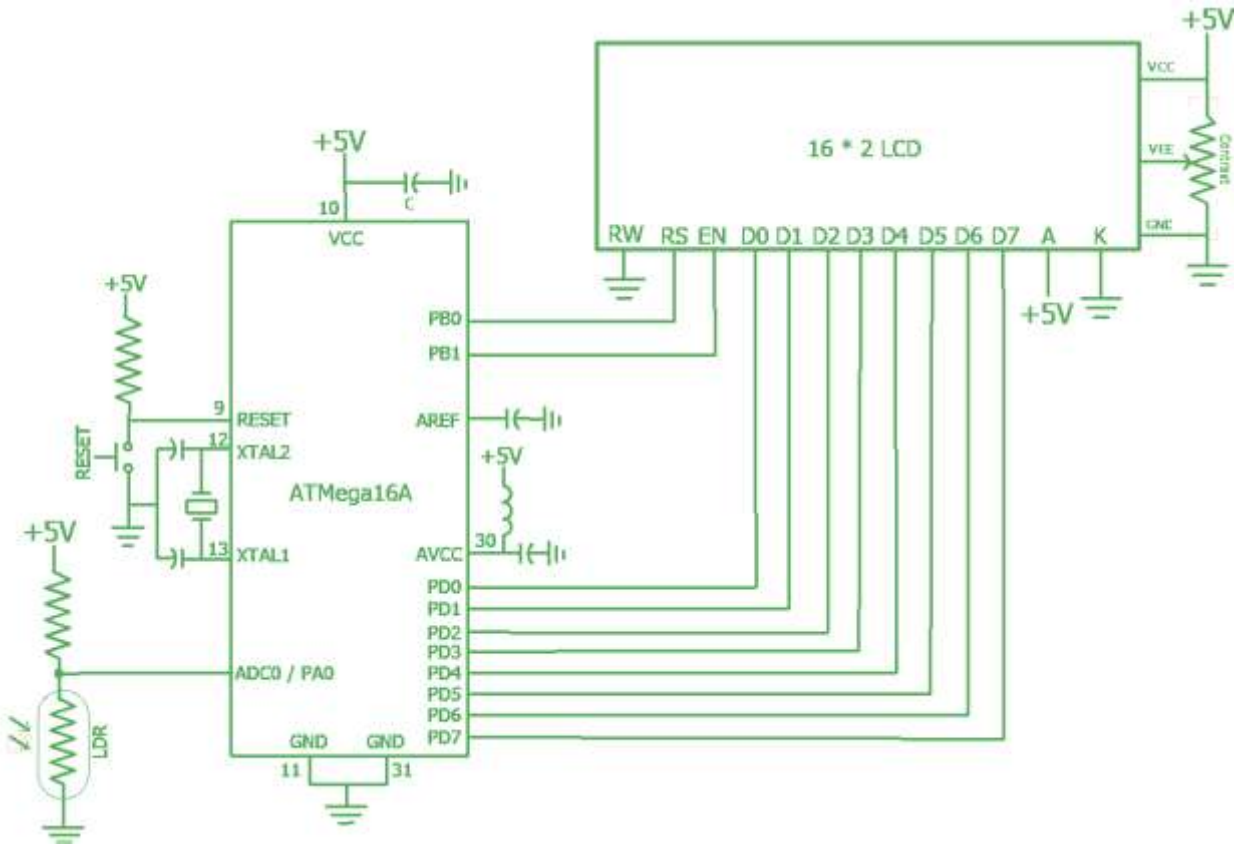
----------**********----------

Experiment – 12

Title: Study of Light intensity measurement using LDR sensor and AVR.

Aim: To study Light intensity measurement using LDR sensor and AVR.

Circuit Diagram:



Theory:

An LDR or light dependent resistor is also known as photo resistor, photocell, and photoconductor. It is a one type of resistor whose resistance varies depending on the amount of light falling on its surface. When the light falls on the resistor, then the resistance changes. These resistors are often used in many circuits where it is required to sense the presence of light. These resistors have a variety of applications. A typical light dependent resistor has a resistance in the darkness of 1MOhm, and in the brightness a resistance of a couple of KOhm.

This resistor works on the principle of photo conductivity. It is nothing but, when the light falls on its surface, then the material conductivity reduces and also the electrons in the valence band of the device are excited to the conduction band. These photons in the incident light must have energy greater than the band gap of the semiconductor material. This makes the electrons to jump from the valence band to conduction. These devices depend on the light, when light falls on the LDR then the resistance decreases, and increases in the dark. When a LDR is kept in the dark place, its resistance is high and, when the LDR is kept in the light its resistance will decrease.

Procedure:

- Start AVR Studio software on PC and write a program.
- Connect the USB cable between USB ASP programmer on the Target board and PC.
- Turn ON the power supply of the target board and then load the .hex file into the AVR using "ProgISP" software.
- Turn OFF the power supply while making circuit connections on the board. Make necessary connections as shown in the circuit diagram.
- Switch ON the power supply of the target board and observe the output.

Program:

```
#include <avr/io.h>
#define F_CPU 4000000UL
#include <util/delay.h>
#include <stdlib.h> // Required for itoa()

void adc_init();
int read_adc(char channel);
void lcd_init();
void lcd_cmd(unsigned char value);
void lcd_data(unsigned char value);
void lcd_print(char *str);

int main() {
 unsigned int adc_value;
 char display_buffer[6];
 // Initialize ADC & LCD
 adc_init();
 lcd_init();
 DDRC = 0xFF;  //make PORTC as o/p
 while (1) {
        // Read ADC channel 0
        adc_value = read_adc(0);
        adc_value = 1024 - adc_value;
```

```
        if(adc_value > 600)
        {
                lcd_cmd(0x80);
                lcd_cmd(0x01);
                lcd_print("Illuminated");
                PORTC = 0X00;  // LED OFF
        }
        else
        {
                lcd_cmd(0x80);
                lcd_cmd(0x01);
                lcd_print("Dark");
                PORTC = 0XFF;  // LED ON
        }
        lcd_cmd(0xC0);
        lcd_print("O/P=");
        itoa(adc_value, display_buffer, 10); // Convert to base 10
        lcd_print(display_buffer);
        _delay_ms(300);
 }
}

void adc_init() {
 DDRA=0x00;  /* Make ADC port as input */
 // Set ADC reference voltage to AVCC
 ADMUX |= (1 << REFS0);
 // Set prescaler to 128
 ADCSRA |= (1 << ADPS2) | (1 << ADPS1) | (1 << ADPS0);
 // Enable ADC
 ADCSRA |= (1 << ADEN);
}
```

```
int read_adc(char channel) {
 // Select ADC channel
 ADMUX = (ADMUX & 0xF0) | (channel & 0x0F);
 // Start conversion
 ADCSRA |= (1 << ADSC);
 while (!(ADCSRA & (1<<ADIF)));  /* Wait until end of conversion by polling ADC interrupt flag */
 ADCSRA |= (1<<ADIF);            /* Clear interrupt flag */
 _delay_ms(1);                   /* Wait a little bit */
 return ADCW;                    /* Return ADC word */
}

void lcd_init()
{
 DDRB = 0xFF;          //make PORTB as o/p
 DDRD = 0xFF;          //make PORTD as o/p
 _delay_ms(50);
 lcd_cmd(0x38);
 lcd_cmd(0x0C);
 lcd_cmd(0x06);
 lcd_cmd(0x01);
 lcd_cmd(0x80);
}

void lcd_cmd(unsigned char value)
{
 PORTD = value;
 PORTB &= 0b11111110;    //RS = 0
 PORTB|=0b00000010;      //EN = 1
 _delay_us(2);
 PORTB &= 0b11111101;    //EN = 0
 _delay_ms(3);
}
```

```
void lcd_data(unsigned char value)
{
 PORTD = value;
 PORTB|=0b00000001;      //RS = 1
 PORTB|=0b00000010;      //EN = 1
 _delay_us(2);
 PORTB &= 0b11111101;    //EN = 0
 _delay_ms(3);
}

void lcd_print(char *str)
{
 unsigned char i;
 for (i = 0; str[i] != '\0'; i++)
 {
        lcd_data(str[i]);
        _delay_ms(3);
 }
}
```

TECHSOLUTION
NEW

Result: Interfacing of LDR to AVR for Light intensity measurement is studied successfully and observe the output.

Teacher's Sign.

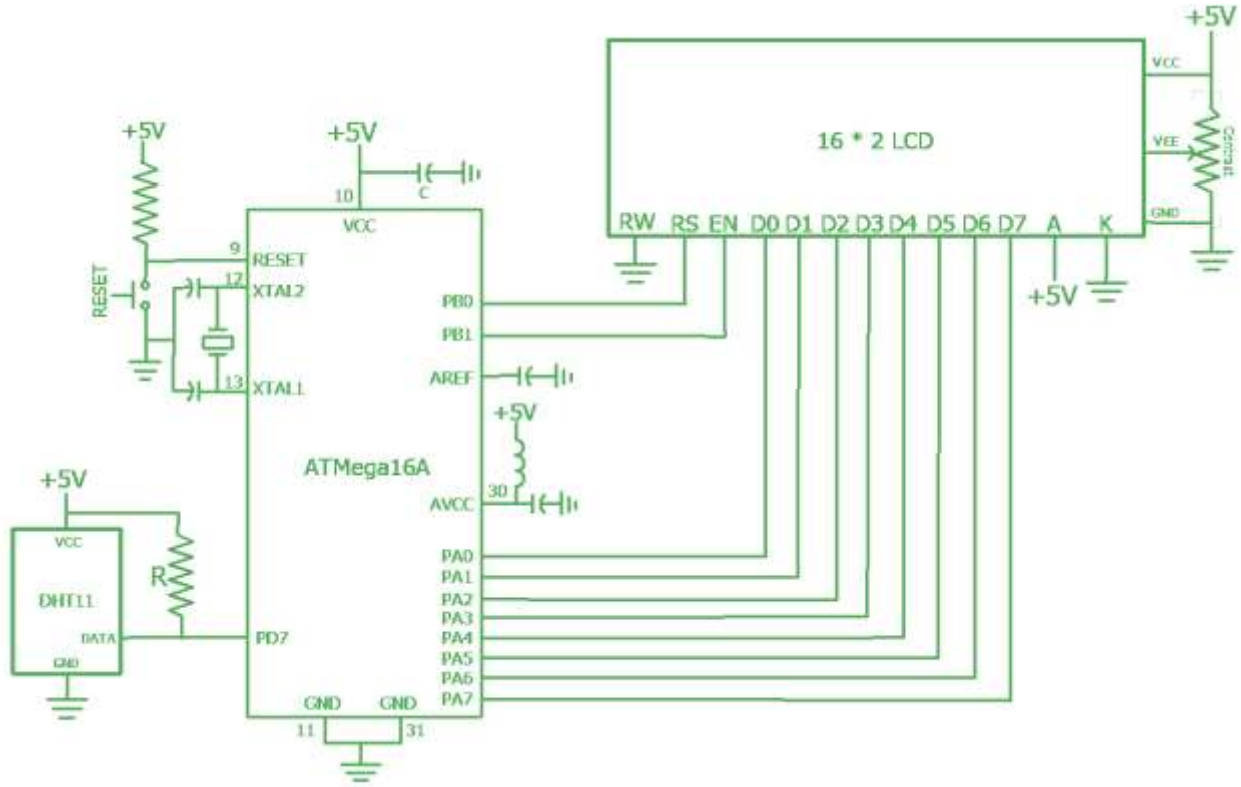
----------**********----------

Experiment – 14

Title: Interfacing DHT11 sensor to AVR Microcontroller

Aim: To study interfacing of DHT11 sensor to AVR and write a program to display temperature and humidity on LCD.

Circuit Diagram:



Theory:

The DHT11 sensor. It is a temperature and humidity sensor having 4 pin single row pin package, single wire serial interface with a calibrated digital signal output. It ensures high reliability, long term stability, excellent quality, fast response, cost effectiveness. The single wire serial interface makes system integration quick and easy. Its small size and low power consumption. It has humidity measurement range of 20% to 90% RH and temperature measurement range from 0 to 50 Celsius.

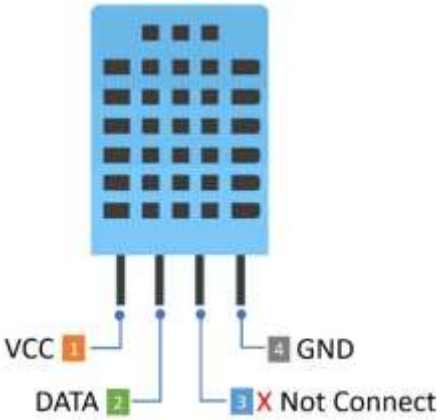


Figure: Pin Diagram of DHT11 Sensor

Procedure:

- Start AVR Studio software on PC and write a program.
- Connect the USB cable between USB ASP programmer on the Target board and PC.
- Turn ON the power supply of the target board and then load the .hex file into the AVR using "ProgISP" software.
- Turn OFF the power supply while making circuit connections on the board. Make necessary connections as shown in the circuit diagram.
- Switch ON the power supply of the target board and observe the output.

Program:

```
#include <avr/io.h>
#include <stdlib.h>
#include <stdio.h>
#define F_CPU 4000000UL
#include <util/delay.h>
#define DHT11_PIN 7   //PD7

void lcd_init();
void lcd_cmd(unsigned char value);
void lcd_data(unsigned char value);
void lcd_print(char *str);

uint8_t c=0,I_RH,D_RH,I_Temp,D_Temp,CheckSum;

void Request()                    /* Microcontroller send start pulse/request */
{
 DDRD |= (1<<DHT11_PIN);
 PORTD &= ~(1<<DHT11_PIN);        /* set to low pin */
 _delay_ms(20);                   /* wait for 20ms */
 PORTD |= (1<<DHT11_PIN);         /* set to high pin */
}
```

```
void Response()                         /* receive response from DHT11 */
{
 DDRD &= ~(1<<DHT11_PIN);
 while(PIND & (1<<DHT11_PIN));
 while((PIND & (1<<DHT11_PIN))==0);
 while(PIND & (1<<DHT11_PIN));
}

uint8_t Receive_data()                  /* receive data */
{
 for (int q=0; q<8; q++)
  {
        while((PIND & (1<<DHT11_PIN)) == 0);  /* check received bit 0 or 1 */
        _delay_us(30);
        if(PIND & (1<<DHT11_PIN))/* if high pulse is greater than 30ms */
        c = (c<<1)|(0x01);      /* then its logic HIGH */
        else                    /* otherwise its logic LOW */
        c = (c<<1);
        while(PIND & (1<<DHT11_PIN));
  }
 return c;
}

int main(void)
{
 char data[5];
 lcd_init();                    /* Initialize LCD */
 lcd_cmd(0x01);                 /* Clear LCD */
 lcd_cmd(0x80);                 /* Enter column and row position */
 lcd_print("Humidity =");
 lcd_cmd(0xC0);
 lcd_print("Temp = ");
```

```
while(1)
{
        Request();              /* send start pulse */
        Response();             /* receive response */
        I_RH=Receive_data();        /* store first eight bit in I_RH */
        D_RH=Receive_data();        /* store next eight bit in D_RH */
        I_Temp=Receive_data();      /* store next eight bit in I_Temp */
        D_Temp=Receive_data();      /* store next eight bit in D_Temp */
        CheckSum=Receive_data();/* store next eight bit in CheckSum */
        if ((I_RH + D_RH + I_Temp + D_Temp) != CheckSum)
        {
                lcd_cmd(0x80);
                lcd_print("Error");
        }
        else
        {
                itoa(I_RH,data,10);
                lcd_cmd(0x8B);
                lcd_print(data);
                lcd_print(".");
                itoa(D_RH,data,10);
                lcd_print(data);
                lcd_print("%");
                itoa(I_Temp,data,10);
                lcd_cmd(0xC6);
                lcd_print(data);
                lcd_print(".");
                itoa(D_Temp,data,10);
                lcd_print(data);
                lcd_data(0xDF);
                lcd_print("C ");
        }
```

TECHSOLUTION NEW

```
        _delay_ms(100);
 }
}

void lcd_init()
{
 DDRB = 0xFF;    //make PORTB as o/p
 DDRA = 0xFF;    //make PORTA as o/p
 _delay_ms(50);
 lcd_cmd(0x38);
 lcd_cmd(0x0C);
 lcd_cmd(0x06);
 lcd_cmd(0x01);
 lcd_cmd(0x80);
}

void lcd_cmd(unsigned char value)
{
 PORTA = value;
 PORTB &= 0b11111110;    //RS = 0
 PORTB|=0b00000010;      //EN = 1
 _delay_us(2);
 PORTB &= 0b11111101;    //EN = 0
 _delay_ms(3);
}
void lcd_data(unsigned char value)
{
 PORTA = value;
 PORTB|=0b00000001;      //RS = 1
 PORTB|=0b00000010;      //EN = 1
 _delay_us(2);
 PORTB &= 0b11111101;    //EN = 0
```

```
 _delay_ms(3);
}

void lcd_print(char *str)
{
 unsigned char i;
 for (i = 0; str[i] != '\0'; i++)
 {
        lcd_data(str[i]);
        _delay_ms(3);
 }
}
```

TECHSOLUTION
NEW

Result: Interfacing of DHT11 sensir to AVR for temperature & humidity measurement is studied successfully and observe the output on LCD.

Teacher's Sign.

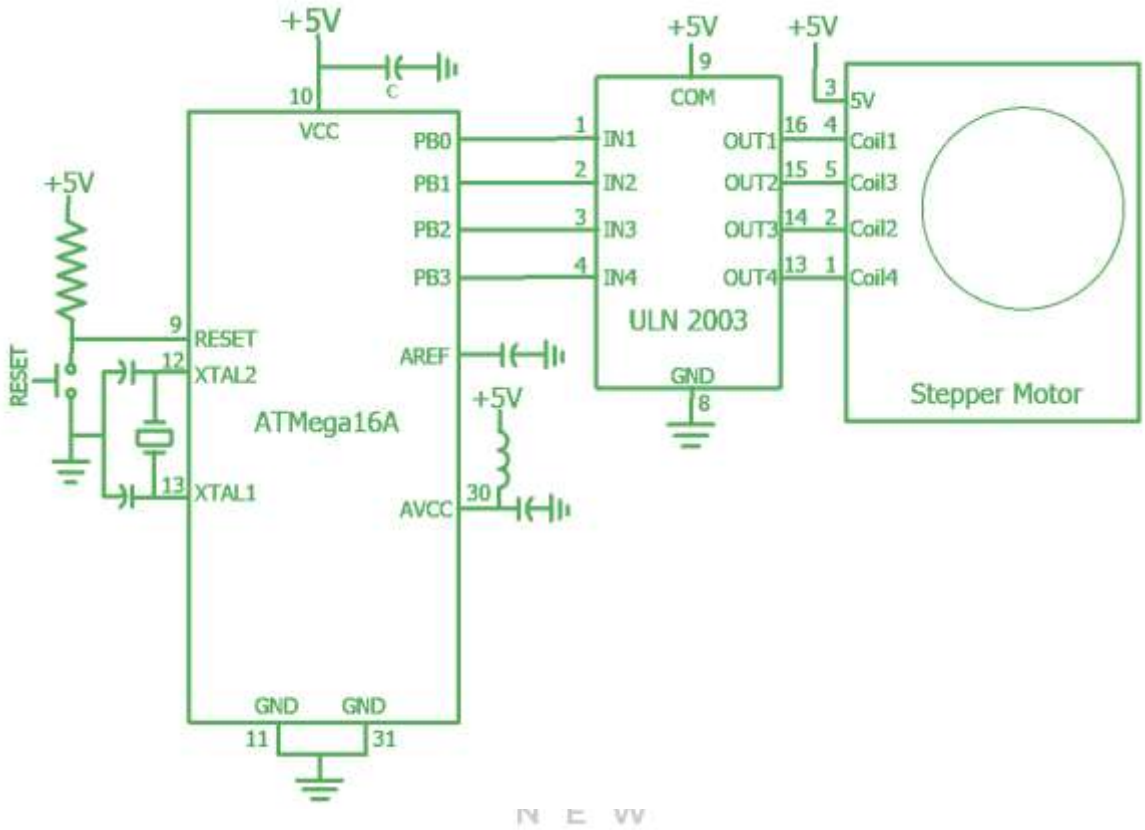
----------**********----------

Experiment – 15

Title: Stepper Motor Interfacing to AVR Microcontroller

Aim: To Study of Stepper Motor interfacing and write a program for clockwise and anticlockwise rotation of motor.

Circuit Diagram:



Theory:

A stepper motor is an electromechanical device which converts electrical pulses into discrete mechanical movements. The shaft or spindle of a stepper motor rotates in discrete step increments when electrical command pulses are applied to it in the proper sequence. The motors rotation has several direct relationships to these applied input pulses. The sequence of the applied pulses is directly related to the direction of motor shafts rotation. The speed of the motor shafts rotation is directly related to the frequency of the input pulses and the length of rotation is directly related to the number of input pulses applied. For this practical stepper motor used is 28byj48. Stepping sequence is 1,2,4,8 for anticlockwise direction and 8,4,2,1 for clockwise direction.

Procedure:

- Start AVR Studio software on PC and write a program.
- Connect the USB cable between USB ASP programmer on the Target board and PC.

- Turn ON the power supply of the target board and then load the .hex file into the AVR using "ProgISP" software.
- Turn OFF the power supply while making circuit connections on the board. Make necessary connections as shown in the circuit diagram.
- Switch ON the power supply of the target board and observe the output.

Program:

```
#include <avr/io.h>
#define F_CPU 4000000UL
#include <util/delay.h>
int main(void)
{
 DDRB=0xFF;    //make PORTB as o/p
 while (1)
 {
        PORTB = 0x08;
        _delay_ms(20);
        PORTB = 0x04;
        _delay_ms(20);
        PORTB = 0x02;
        _delay_ms(20);
        PORTB = 0x01;
        _delay_ms(20);
 }
}
```

Note: Change the sequence to 1,2,4,8 from 8,4,2,1 to rotate motor in anticlockwise direction.

Result: Interfacing of stepper motor with AVR is successfully studied and observe the output.

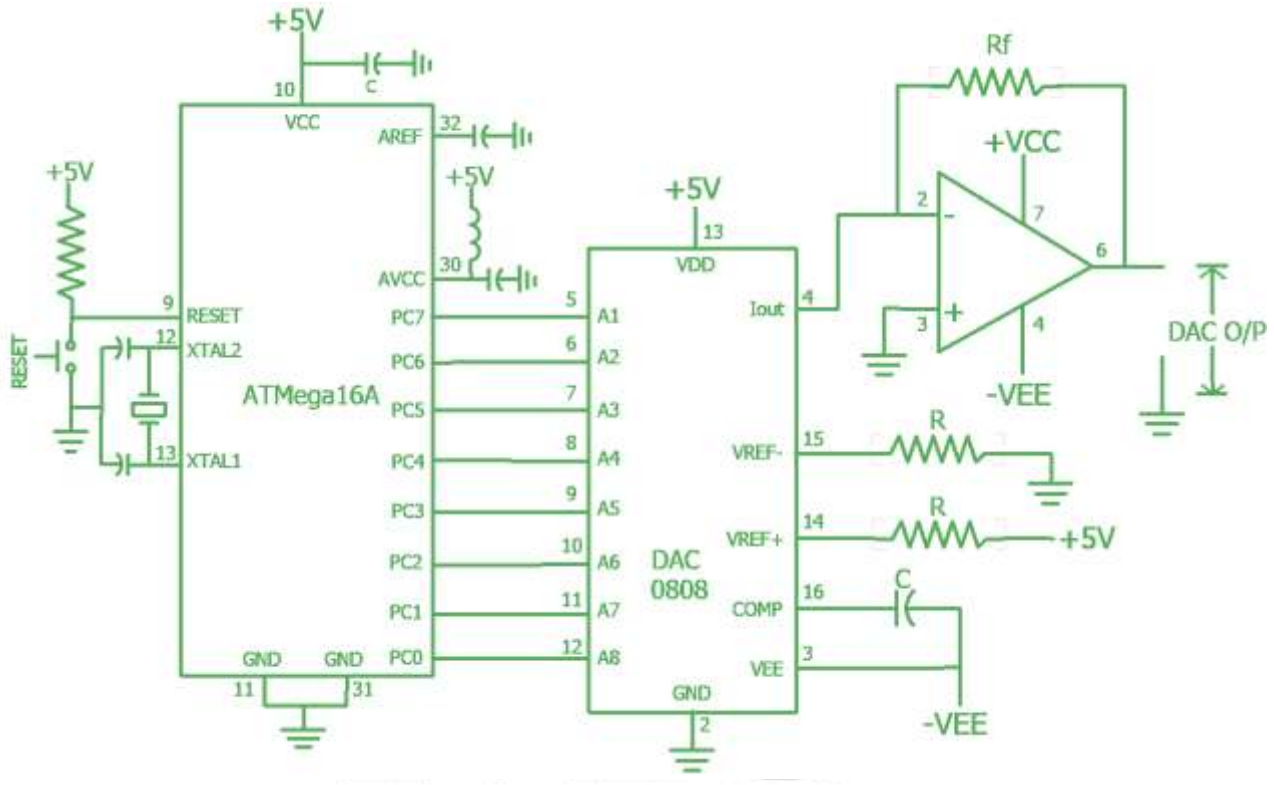
Teacher's Sign.

Experiment – 16

Title: Waveform Generation using DAC

Aim: To study and understand interfacing of DAC 0808 to AVR microcontroller and generate different waveforms.

Circuit Diagram:



Theory:

In electronics, a digital-to-analog converter (DAC) is a device that converts a digital signal into an analog signal. There are several DAC architectures; the suitability of a DAC for a particular application is determined by three main parameters: resolution, maximum sampling frequency and accuracy. Due to the complexity and the need for precisely matched components, all but the most specialized DACs are implemented as integrated circuits (ICs). Digital-to-analog conversion can degrade a signal, so a DAC should be specified that has insignificant errors in terms of the application. There are two types of DAC's, R-2R ladder DAC and binary weighted DAC.

Introduction to DAC0808 - The DAC0808 is an 8-bit monolithic digital-to-analog converter (DAC) featuring a full scale output current. The DAC0808 will interface directly with popular TTL, DTL or CMOS logic levels. It is R-2R type DAC.

- Relative accuracy: ±0.19% error maximum.
- Fast settling time: 150 ns typ.
- Non-inverting digital inputs are TTL and CMOS compatible.

- Power supply voltage range: ±4.5V to ±18V.
- Low power consumption: 33 mW @ ±5V.

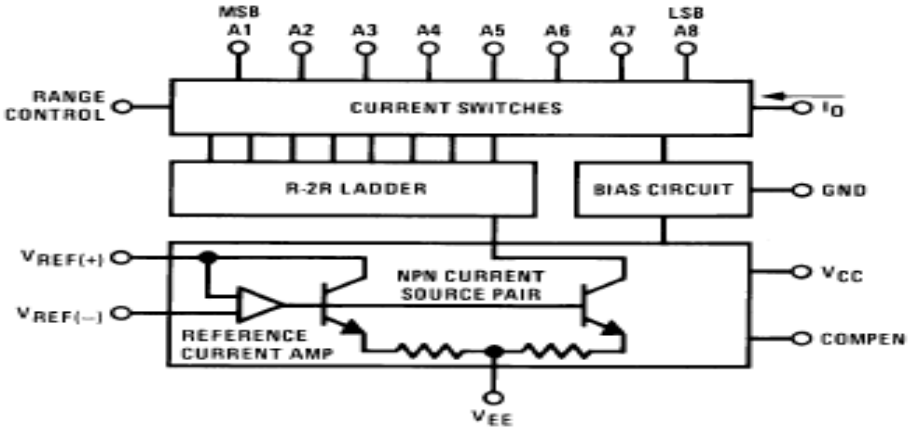


Figure: Internal diagram of DAC 0808

Procedure:

- Start AVR Studio software on PC and write a program.
- Connect the USB cable between USB ASP programmer on the Target board and PC.
- Turn ON the power supply of the target board and then load the .hex file into the AVR using "ProgISP" software.
- Turn OFF the power supply while making circuit connections on the board. Make necessary connections as shown in the circuit diagram.
- Switch ON the power supply of the target board and observe the output.

Program:

Square Wave –

```
#include <avr/io.h>
#define F_CPU 4000000UL
#include <util/delay.h>
int main(void)
{
 MCUCSR|=(1<<JTD);  /*diable JTAG port*/
 MCUCSR|=(1<<JTD);
 DDRC=0xFF;    //make PORTC as o/p
 while (1)
 {
        PORTC = 0x00;
        _delay_ms(50);
```

```
        PORTC = 0xFF;
        _delay_ms(50);
 }
}
```

Sawtooth / Ramp Waveform –

```
#include <avr/io.h>
#define F_CPU 4000000UL
#include <util/delay.h>
int main(void)
{
 MCUCSR|=(1<<JTD);  /*diable JTAG port*/
 MCUCSR|=(1<<JTD);
 char i;
 DDRC=0xFF;    //make PORTC as o/p
 while (1)
 {
        for(i=0;i<255;i++)
        {
                PORTC = i;
        }
 }
}
```

Triangular Waveform –

```
#include <avr/io.h>
#define F_CPU 4000000UL
#include <util/delay.h>
int main(void)
{
 MCUCSR|=(1<<JTD);  /*diable JTAG port*/
 MCUCSR|=(1<<JTD);
 char i;
 DDRC=0xFF;    //make PORTC as o/p
```

```
while (1)
{
        for(i=0;i<255;i++)
        {
                PORTC = i;
        }
        for(i=255;i>0;i--)
        {
                PORTC = i;
        }
}
}
```

TECHSOLUTION
NEW

Result: Interfacing of DAC with AVR is successfully studied and different waveforms were generated successfully.

Teacher's Sign.

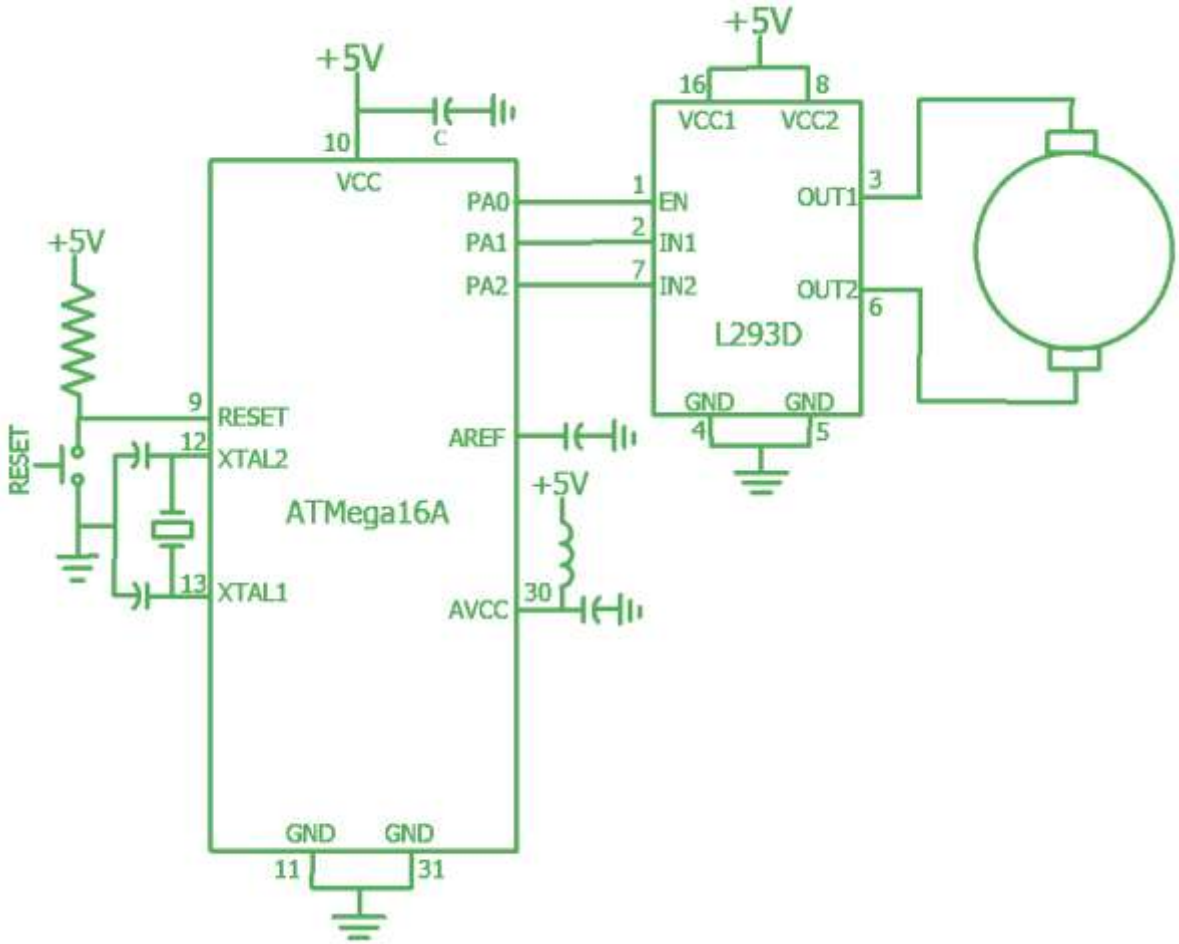
----------**********----------

Experiment – 17

Title: DC Motor Interfacing to AVR Microcontroller

Aim: To study and understand DC motor interfacing & write a program to vary speed of DC motor using PWM.

Circuit Diagram:



Theory:

A DC motor is any of a class of rotary electrical machines that converts direct current electrical power into mechanical power. The most common types rely on the forces produced by magnetic fields. Nearly all types of DC motors have some internal mechanism, either electromechanical or electronic, to periodically change the direction of current flow in part of the motor. DC motors were the first type widely used, since they could be powered from existing direct-current lighting power distribution systems. A DC motor's speed can be controlled over a wide range, using either a variable supply voltage or by changing the strength of current in its field windings.

Introduction to IC L293D - The Device is a monolithic integrated high voltage, high current four channel driver designed to accept standard DTL or TTL logic levels and drive inductive loads (such as relays solenoides, DC and stepping motors) and switching power transistors. To simplify use as two

bridges each pair of channels is equipped with an enable input. A separate supply input is provided for the logic, allowing operation at a lower voltage and internal clamp diodes are included.

Procedure:

- Start AVR Studio software on PC and write a program.
- Connect the USB cable between USB ASP programmer on the Target board and PC.
- Turn ON the power supply of the target board and then load the .hex file into the AVR using "ProgISP" software.
- Turn OFF the power supply while making circuit connections on the board. Make necessary connections as shown in the circuit diagram.
- Switch ON the power supply of the target board and observe the output.

Program:

For Clockwise rotation -

```
#include <avr/io.h>
#define F_CPU 4000000UL
#include <util/delay.h>
//PA1=IN1, PA2=IN2, PA0= EN
int main()
{
 DDRA=0xFF;                    //make op
 PORTA |= (1<<PA1);            //IN1 = 1
 PORTA &= (~(1<<PA2));         //IN2 = 0
 while(1)
 {
        PORTA |= 0b00000001;          //PA0= HIGH
        _delay_ms(30);
        PORTA &= 0b11111110;          //PA0 = LOW
        _delay_ms(40);
 }
}
```

For Anti-clockwise rotation -

```
#include <avr/io.h>
#define F_CPU 4000000UL
#include <util/delay.h>
//PA1=IN1, PA2=IN2, PA0= EN
int main()
{
 DDRA=0xFF;                    //make op
 PORTA |= (1<<PA2);            //IN2 = 1
 PORTA &= (~(1<<PA1));         //IN1 = 0
 while(1)
 {
        PORTA |= 0b00000001;          //PA0= HIGH
        _delay_ms(30);
        PORTA &= 0b11111110;          //PA0 = LOW
        _delay_ms(40);
 }
}
```

Result: Interfacing of DC motor with AVR is successfully studied and observe the output.

Teacher's Sign.